

Universidade Federal de Minas Gerais
Escola de Engenharia
Curso de Graduação em Engenharia de Controle e Automação

**Monitoramento e controle de processos para o
consumo de energia residencial baseado em Sistemas
Inteligentes**

Igor Cleto Silva de Araújo

Orientadora: Prof Carmela Maria Polito Braga

Belo Horizonte, Outubro de 2024

Monografia

Monitoramento e controle de processos para o consumo de energia residencial baseado em Sistemas Inteligentes

Monografia submetida à banca examinadora designada pelo Colegiado Didático do Curso de Graduação em Engenharia de Controle e Automação da Universidade Federal de Minas Gerais, como parte dos requisitos para aprovação na atividade Projeto Final de Curso II.

Belo Horizonte, Outubro de 2024

Resumo

Este projeto tem como objetivo desenvolver um sistema inteligente para monitoramento de energia utilizando controle estatístico de processo (CEP) em residências que não foram projetadas originalmente para automação residencial. Especificamente, visa realizar uma pesquisa e análise de tecnologias existentes que permitam o monitoramento energético sem a necessidade de infraestrutura prévia, explorar soluções adequadas para residências convencionais, desenvolver um protótipo de sistema de monitoramento de fácil instalação e uso, implementar algoritmos de processamento de dados e controle estatístico de processos (CEP) para o monitoramento de energia, criar uma plataforma de software intuitiva para visualização e análise dos dados pelos usuários, testar e validar o sistema em ambiente real para garantir sua eficiência e usabilidade, promover a viabilidade de implementação em larga escala por meio da análise de custos e estratégias de adoção, e documentar e divulgar os resultados obtidos, destacando as contribuições para a Engenharia de Controle e Automação e os benefícios sociais e ambientais proporcionados pelo sistema desenvolvido.

Abstract

This project aims to develop an intelligent system for monitoring energy consumption using statistical process control (SPC) in homes that were not originally designed for home automation. Specifically, it seeks to conduct research and analysis of existing technologies that allow energy monitoring without the need for prior infrastructure, explore suitable solutions for conventional residences, develop a prototype of a monitoring system that is easy to install and use, implement data processing algorithms and statistical process control (SPC) for energy monitoring, create an intuitive software platform for data visualization and analysis by users, test and validate the system in a real environment to ensure its efficiency and usability, promote the feasibility of large-scale implementation through cost analysis and adoption strategies, and document and disseminate the results obtained, highlighting the contributions to Control and Automation Engineering and the social and environmental benefits provided by the developed system.

Agradecimentos

Gostaria de expressar minha profunda gratidão a Deus pela saúde física e mental que me permitiram realizar meus sonhos e objetivos. Aos meus pais, meu sincero agradecimento pelo suporte incondicional aos meus estudos desde a infância, sempre acreditando em meu potencial e me incentivando a seguir em frente. Agradeço também aos meus professores e gestores que, com seu apoio e ensinamentos, tanto dentro quanto fora da sala de aula, contribuíram para a minha formação acadêmica e pessoal. Não poderia deixar de agradecer, de maneira especial, à minha orientadora, Prof^a Carmela Maria Polito Braga, pela dedicação, paciência e orientação ao longo deste ano de desenvolvimento do trabalho. Sua contribuição foi fundamental para a realização deste projeto. A todos, o meu muito obrigado.

Sumário

Resumo	i
Abstract	iii
Agradecimentos	v
Lista de Figuras	xi
Lista de Tabelas	xiii
1 Introdução	1
1.1 Motivação e Justificativa	1
1.2 Objetivos do Projeto	2
1.3 Local de Realização	2
1.4 Estrutura da Monografia	3
2 Fundamentos da Automação Residencial	5
2.1 Arquitetura de um Sistema de Automação Residencial	6
2.1.1 Sensores	6
2.1.2 Atuadores	7
2.1.3 Controladores	8
2.1.4 Tecnologias de Comunicação	8
2.1.5 Interface de Usuário	10
2.1.6 Serviços em Nuvem e Análise de Dados	11
2.1.7 Segurança e Privacidade	13
2.1.8 Integração com Outros Sistemas	15
2.1.9 Escalabilidade e Flexibilidade	16
2.1.10 Instrumentação do Processo	18
2.2 Monitoramento Energético	20
2.2.1 Coleta e Organização dos Dados	21

2.2.2	Pré-Processamento, Qualidade dos Dados e Controle Estatístico de Processo	21
2.2.3	Análise Estatística e Inteligência Artificial	22
2.2.4	Privacidade, LGPD e Proteção dos Dados Energéticos	23
2.2.5	Integração com Outros Sistemas e Evolução Contínua	24
2.2.6	Benefícios Práticos e Responsabilidade Social	24
2.3	Modelos Arquiteturais para Sistemas de Automação Residencial e Monitoramento Energético	25
2.3.1	Visão Geral de um Sistema Típico	25
2.3.2	Camada de Aquisição de Dados	25
2.3.3	Camada de Processamento e Gerenciamento de Dados	26
2.3.4	Camada de Apresentação e Interação com o Usuário	27
2.3.5	Lógica de Automação e Agendamento de Rotinas	28
3	Metodologia	29
3.1	Pesquisa e Análise de Tecnologias Existentes	29
3.1.1	Seleção de Ferramentas de Software e Plataformas de Desenvolvimento	34
3.2	Desenvolvimento do Protótipo	44
3.2.1	Seleção e Configuração de Sensores	44
3.2.2	Arquitetura e Desenvolvimento do Software de Coleta, Processamento e Controle	47
3.2.3	Desenvolvimento da Interface de Usuário com Streamlit	55
3.3	Implementação de Algoritmos de Processamento e Análise	58
3.3.1	Pré-processamento dos Dados	58
3.3.2	Análise Estatística e Controle Estatístico de Processos (CEP)	60
3.4	Validação em Ambiente Real	64
3.5	Viabilidade de Implementação em Larga Escala	66
3.6	Resumo do Capítulo	66
4	Resultados	67
4.1	Atividades do Projeto	67
4.2	Requisitos do Sistema	67
4.3	Desenvolvimento e Implementação	67
4.4	Testes	67
4.5	Análise dos Resultados	67
4.6	Resumo do Capítulo	68
4.7	Formato, expressões matemáticas e o L ^A T _E X	68

<i>SUMÁRIO</i>	ix
4.7.1 O L ^A T _E X	68
4.7.2 Expressões Matemáticas	69
5 Conclusões	73
5.1 Considerações Finais	73
5.2 Propostas de Continuidade	73
A O que ficou para depois	75
Referências Bibliográficas	74
B O que mais faltou	77

Lista de Figuras

3.1	Fluxograma da pipeline de processamento de dados de consumo energético com Dagster.	50
3.2	Fluxograma da pipeline de agendamento e execução de cenas com Dagster.	54
3.3	Fluxograma da navegação geral da aplicação Streamlit.	55
3.4	Fluxograma da lógica de controle individual de dispositivo na aplicação Streamlit.	57
3.5	Fluxograma do processo de criação de cenas na aplicação Streamlit.	59

Lista de Tabelas

4.1	Exemplo de tabela - Coloque toda informação sobre a tabela aqui . . .	67
-----	---	----

Capítulo 1

Introdução

1.1 Motivação e Justificativa

A gestão eficiente do consumo de energia elétrica em ambientes residenciais representa um desafio contemporâneo crucial, impulsionado tanto pela crescente demanda energética global quanto pela urgência na adoção de práticas sustentáveis. Frequentemente, os usuários domésticos carecem de visibilidade detalhada sobre seus perfis de consumo e de ferramentas acessíveis para o gerenciamento ativo, o que cria obstáculos para a identificação de desperdícios e a adoção de medidas eficazes para a otimização energética e redução de custos. Essa lacuna é exacerbada pela escassez de sistemas de monitoramento e controle que sejam simultaneamente de fácil implementação em residências convencionais e intuitivos para o usuário final, dificultando a transição para um uso mais consciente e econômico da energia no cotidiano.

Paralelamente, o campo da automação residencial, intrinsecamente ligado à gestão energética inteligente, confronta-se com obstáculos significativos de interoperabilidade. A heterogeneidade de dispositivos, sensores e tecnologias de comunicação, provenientes de múltiplos fabricantes e frequentemente baseados em padrões proprietários ou concorrentes, resulta em ecossistemas fragmentados. Essa fragmentação impede a comunicação eficiente entre componentes e a consequente criação de sistemas de monitoramento e controle verdadeiramente unificados e coesos. Tal cenário não apenas subutiliza o potencial da automação para otimizar o consumo energético e elevar a qualidade de vida, mas também restringe o acesso a soluções tecnológicas que poderiam fomentar ambientes domésticos mais confortáveis, seguros e ambientalmente responsáveis.

Agrava este panorama o fato de que uma vasta parcela do parque imobiliário residencial existente não foi concebida com infraestrutura para automação. A modernização desses lares para incorporar tecnologias de monitoramento e controle energético inteligente frequentemente implica intervenções estruturais complexas e custos proibiti-

vos para muitos proprietários. Essa barreira à adoção, tanto técnica quanto financeira, priva um número expressivo de residências dos benefícios tangíveis da automação — como maior eficiência energética, conforto aprimorado e segurança reforçada. Consequentemente, a carência de soluções que sejam simultaneamente acessíveis, de fácil instalação em edificações convencionais e eficazes na gestão do consumo energético configura uma oportunidade premente para o desenvolvimento de inovações capazes de transpor esses obstáculos e democratizar o acesso a um gerenciamento energético residencial mais inteligente e sustentável.

1.2 Objetivos do Projeto

Tendo em vista o exposto acima, este projeto tem por objetivos:

- a) Realizar uma pesquisa e análise de tecnologias existentes que permitam o monitoramento energético sem a necessidade de infraestrutura prévia.
- b) Explorar soluções adequadas para residências convencionais, que não foram originalmente projetadas para automação residencial.
- c) Desenvolver um protótipo de sistema de monitoramento e controle de fácil instalação e uso, focado na aplicação de princípios de controle estatístico de processo.
- d) Implementar algoritmos para processamento de dados, previsão de consumo e aplicação de técnicas de controle estatístico de processo.
- e) Desenvolver uma plataforma de software intuitiva para visualização, análise dos dados e suporte ao controle estatístico de processo pelo usuário.
- f) Testar e validar o sistema em ambiente real, garantindo sua eficiência e usabilidade.
- g) Promover a viabilidade de implementação em larga escala por meio da análise de custos e estratégias de adoção.
- h) Documentar e divulgar os resultados obtidos, destacando as contribuições para a Engenharia de Controle e Automação e os benefícios sociais e ambientais proporcionados pelo sistema desenvolvido.

1.3 Local de Realização

O local de realização deste projeto é a residência do próprio desenvolvedor, onde os testes dos equipamentos e a simulação das necessidades reais de automação residencial

estão sendo realizados. A escolha deste ambiente se justifica pela possibilidade de realizar uma avaliação prática e realista das soluções de monitoramento e controle estatístico de processos para o consumo energético, simulando as condições do dia a dia em uma residência convencional. A residência não foi originalmente projetada com infraestrutura para automação, o que representa um desafio adicional, mas também um aspecto relevante para a validação do sistema, pois permite testar a adaptação de tecnologias em um ambiente que não conta com recursos avançados de automação.

Neste contexto, o projeto visa avaliar a viabilidade de implementação de soluções de monitoramento energético em ambientes residenciais que, assim como muitas casas, não foram concebidos para suportar sistemas automatizados. Dessa forma, o projeto busca não apenas desenvolver uma solução eficiente, mas também uma que seja de fácil adaptação e acessível para a grande maioria dos consumidores. O vínculo com o local de testes é direto, pois, além de ser o ambiente de realização das simulações, a residência do desenvolvedor também serve como um campo de experimentação e validação para os resultados do sistema proposto.

1.4 Estrutura da Monografia

O trabalho está dividido em quatro capítulos. Este capítulo apresentou uma introdução ao projeto descrito nesta monografia, incluindo o objetivo de desenvolvimento de um sistema inteligente para monitoramento e controle de processos para o consumo de energia em residências, além de descrever o local de realização dos testes, que ocorre na residência do próprio desenvolvedor. O Capítulo 2 descreve os princípios básicos de um sistema de automação residencial e monitoramento energético, e detalha modelos arquiteturais genéricos para tais sistemas, abordando as camadas de aquisição de dados, processamento, gerenciamento, apresentação e automação. O Capítulo 3 explora a metodologia de desenvolvimento do protótipo proposto, incluindo a análise de tecnologias, a criação do protótipo, a implementação de uma pipeline de dados para coleta e processamento de informações energéticas, o desenvolvimento de uma aplicação para interação do usuário e a implementação dos algoritmos de processamento de dados e previsão de consumo. No Capítulo 4, apresenta-se a conclusão da monografia, com um resumo dos resultados alcançados e algumas sugestões e dificuldades encontradas durante a realização do projeto.

Capítulo 2

Fundamentos da Automação Residencial e Monitoramento Energético

Este capítulo apresenta uma visão abrangente dos princípios fundamentais na concepção, implementação e operação de um sistema inteligente de automação residencial orientado ao monitoramento e previsão do consumo energético. O contexto abordado parte da realidade de residências convencionais, isto é, aquelas que não dispõem de infraestrutura prévia de automação, exigindo assim soluções flexíveis, escaláveis e capazes de integrar múltiplos dispositivos e tecnologias. Nosso enfoque não se limita ao simples acionamento de equipamentos, mas abrange a coleta e análise detalhada de dados, permitindo uma gestão mais eficiente dos recursos, redução de custos, melhoria do conforto e incremento da sustentabilidade ambiental.

Iniciamos com um exame minucioso da arquitetura típica de um sistema de automação residencial, descrevendo os principais componentes: sensores (ambientais, de presença, de consumo energético), atuadores (termostatos, dimmers, motores para cortinas, entre outros), controladores (centralizados e distribuídos), as tecnologias de comunicação que viabilizam a interconexão desses elementos, e as interfaces de usuário (aplicativos, painéis físicos, assistentes de voz). A interoperabilidade entre dispositivos de diferentes fabricantes é um desafio central, e esforços de padronização buscam garantir maior flexibilidade, segurança e escalabilidade ao ecossistema doméstico conectado.

Em seguida, detalhamos o processo de instrumentação do sistema, abordando a seleção criteriosa de hardware, a integração de sensores e atuadores, bem como a definição da topologia da rede. Esse cuidado assegura a qualidade dos dados coletados, a robustez do controle e a adequação às normas legais. A segurança da informação e a privacidade são tratadas de forma transversal, observando-se os princípios da Lei Geral

de Proteção de Dados (LGPD) e adotando medidas como criptografia, autenticação de dispositivos, minimização de dados e consentimento informado dos usuários.

Outro ponto central é o monitoramento energético. Não se trata apenas de registrar o consumo, mas de transformar dados brutos em conhecimento. São exploradas técnicas de pré-processamento, análise estatística, aprendizado de máquina e Controle Estatístico de Processo (CEP) para detectar anomalias, identificar padrões de consumo, antecipar demandas e propor intervenções que otimizem o uso de energia. Essas abordagens analíticas embasam decisões proativas, possibilitando desde ajustes automáticos de cargas elétricas até a recomendação de substituição de equipamentos ineficientes, o que resulta em benefícios econômicos, maior conforto e redução do impacto ambiental.

Ao compreender as soluções existentes, suas limitações e potenciais, este capítulo fornece uma base sólida para os desenvolvimentos subsequentes, orientando tanto a implementação prática do protótipo quanto a evolução do sistema no sentido de maior inteligência, integração e respeito à privacidade e às leis vigentes.

2.1 Arquitetura de um Sistema de Automação Residencial

A arquitetura de um sistema de automação residencial é composta por diversos elementos que trabalham em conjunto para oferecer controle, monitoramento e otimização de aspectos como eficiência energética, segurança, conforto e conveniência. Ela não consiste apenas em dispositivos isolados, mas em um ecossistema integrado capaz de entender as condições do ambiente, agir de forma proativa e se adaptar às preferências dos moradores.

A seguir, detalhamos os principais elementos dessa arquitetura, estabelecendo as bases para compreender as complexidades e potencialidades de tais sistemas.

2.1.1 Sensores

Os sensores representam a interface entre o mundo físico e o digital. Eles coletam dados sobre o ambiente e o estado dos dispositivos, fornecendo informações essenciais para que o sistema possa tomar decisões informadas. Podem monitorar temperatura, umidade, luminosidade, qualidade do ar, presença de pessoas e consumo de energia, entre outros parâmetros. Alguns exemplos:

- **Sensores de movimento e presença:** Utilizam tecnologias como infravermelho passivo (PIR), ultrassom ou micro-ondas. Além de acionar luzes ou alarmes,

podem colaborar na climatização, ajustando a temperatura apenas em ambientes ocupados.

- **Sensores ambientais (temperatura, umidade, qualidade do ar):** Auxiliam no controle de sistemas de HVAC (Aquecimento, Ventilação e Ar Condicionado), garantindo conforto térmico e qualidade do ar interno.
- **Sensores de luminosidade:** Ajustam iluminação artificial conforme a luz natural, otimizando o uso de energia e melhorando o bem-estar.
- **Sensores de consumo energético:** Medem o consumo de dispositivos específicos ou circuitos, fornecendo dados granulares para análise, detecção de aparelhos ineficientes e implementação de estratégias de economia de energia.

A seleção e instalação adequadas dos sensores são fundamentais, influenciando a qualidade dos dados coletados e a efetividade das estratégias de controle.

2.1.2 Atuadores

Atuadores transformam as decisões do sistema em ações físicas. Podem ser relés, motores, dimmers ou válvulas controladas eletronicamente, acionando iluminação, climatização, cortinas, persianas, eletrodomésticos e outros dispositivos.

- **Interruptores e dimmers inteligentes:** Ligam, desligam ou ajustam a intensidade luminosa, criando cenários adequados para diferentes atividades.
- **Termostatos inteligentes:** Regulam o aquecimento ou resfriamento, aprendendo rotinas e antecipando demandas, reduzindo custos e melhorando o conforto.
- **Motores para cortinas e persianas:** Controlam entrada de luz e calor, integrando-se a sensores de luminosidade e temperatura para otimizar a eficiência energética.
- **Tomadas inteligentes:** Permitem ligar e desligar aparelhos remotamente, fornecendo dados de consumo e contribuindo para reduzir o uso excessivo de energia.

A qualidade, confiabilidade e responsividade dos atuadores são essenciais para garantir que as decisões do sistema sejam efetivamente postas em prática.

2.1.3 Controladores

Os controladores são o “cérebro” do sistema, processando as informações dos sensores e enviando comandos aos atuadores. Podem ser centralizados ou distribuídos, e sua complexidade varia desde microcontroladores simples até sistemas embarcados sofisticados e servidores locais.

- **Controlador Centralizado:** Uma unidade concentra a lógica, facilitando a manutenção, porém criando um ponto único de falha.
- **Controladores Distribuídos:** Cada área ou cômodo pode ter seu próprio controlador, tornando o sistema mais robusto e escalável.
- **Inteligência Artificial e Aprendizado de Máquina:** Permitem o ajuste dinâmico dos parâmetros de controle com base em dados históricos, antecipação de demandas e identificação de padrões complexos.

A escolha do controlador depende do nível de inteligência desejado, da escalabilidade necessária e da capacidade de processar algoritmos avançados de previsão e otimização.

2.1.4 Tecnologias de Comunicação

A comunicação eficaz entre sensores, atuadores, controladores e interfaces de usuário é um pilar fundamental da automação residencial. A escolha das tecnologias de comunicação impacta diretamente o desempenho, a confiabilidade, o consumo energético e a escalabilidade do sistema. Em vez de focar em protocolos específicos, que evoluem rapidamente, é mais produtivo compreender os princípios e características que norteiam essas escolhas.

As tecnologias de comunicação em automação residencial podem ser amplamente categorizadas com base em diversos fatores:

- **Meio Físico:** As comunicações podem ser cabeadas (ex: Ethernet, par trançado dedicado) ou sem fio (ex: rádio frequência - RF, infravermelho). Soluções sem fio são predominantes em residências existentes devido à facilidade de instalação, enquanto soluções cabeadas podem oferecer maior confiabilidade e largura de banda em novas construções ou reformas profundas.
- **Alcance:** Diferentes tecnologias oferecem alcances variados, desde alguns metros (ideal para comunicação dentro de um mesmo cômodo) até dezenas ou centenas de metros (para cobrir toda a residência e áreas externas).

- **Largura de Banda (Taxa de Dados):** A quantidade de dados que pode ser transmitida por unidade de tempo varia significativamente. Aplicações como streaming de vídeo exigem alta largura de banda, enquanto sensores simples (temperatura, presença) necessitam de taxas muito menores.
- **Consumo Energético:** Este é um fator crítico, especialmente para dispositivos alimentados por bateria, como sensores sem fio. Tecnologias de baixo consumo são essenciais para garantir longa autonomia e reduzir a manutenção.
- **Topologia de Rede:** A forma como os dispositivos são interconectados influencia a robustez e a cobertura da rede.
 - *Estrela (Star):* Todos os dispositivos se comunicam diretamente com um coordenador central (hub). Simples de gerenciar, mas o coordenador é um ponto único de falha e o alcance é limitado por ele.
 - *Malha (Mesh):* Os dispositivos podem retransmitir mensagens uns para os outros, criando múltiplos caminhos de comunicação. Isso aumenta a robustez (se um nó falha, a mensagem pode encontrar outro caminho) e estende o alcance da rede.
 - *Ponto a Ponto (Peer-to-Peer):* Dispositivos comunicam-se diretamente entre si sem um coordenador central, comum em algumas aplicações de curto alcance.
- **Interferência e Confiabilidade:** Redes sem fio, especialmente aquelas que operam em bandas de frequência congestionadas (como 2.4 GHz), podem sofrer interferência de outros dispositivos (micro-ondas, telefones sem fio, outras redes Wi-Fi). A escolha da frequência e técnicas de modulação podem mitigar esses problemas.
- **Custo:** O custo dos módulos de comunicação e da infraestrutura associada (hubs, roteadores) é um fator importante na viabilidade de um sistema.

Um dos maiores desafios na automação residencial tem sido a **interoperabilidade** – a capacidade de dispositivos de diferentes fabricantes comunicarem-se e trabalharem juntos de forma transparente. Historicamente, a proliferação de protocolos proprietários e padrões concorrentes fragmentou o mercado, exigindo múltiplos hubs e aplicativos para controlar diferentes partes da casa. Esforços de padronização da indústria buscam resolver esse problema, promovendo camadas de aplicação comuns sobre tecnologias de rede IP (Internet Protocol), o que facilita a integração e simplifica a experiência do usuário.

A **segurança** na camada de comunicação é vital. Isso inclui a criptografia dos dados transmitidos para proteger contra espionagem, a autenticação de dispositivos para garantir que apenas componentes autorizados participem da rede, e a integridade das mensagens para prevenir adulterações.

Em resumo, a seleção de tecnologias de comunicação para um sistema de automação residencial envolve uma análise cuidadosa das necessidades específicas da aplicação, considerando o equilíbrio entre alcance, taxa de dados, consumo de energia, confiabilidade, custo e a importância da interoperabilidade e segurança.

2.1.5 Interface de Usuário

A interface de usuário (UI) é o ponto de contato crucial entre os moradores e o sistema de automação residencial, sendo fundamental para a aceitação e o uso efetivo da tecnologia. Uma UI bem projetada deve ser intuitiva, acessível e eficiente, permitindo que os usuários controlem e monitorem suas casas com facilidade, independentemente de seu nível de familiaridade tecnológica. As principais formas de interface incluem:

- **Aplicativos Móveis e Web:** Considerados o principal meio de interação para muitos sistemas, oferecem a conveniência do acesso remoto, permitindo o controle e monitoramento de qualquer lugar com conexão à internet. Funcionalidades comuns incluem a visualização em tempo real do status dos dispositivos, consulta a históricos detalhados de consumo energético e eventos, criação e acionamento de cenários de automação personalizados (e.g., "modo cinema", "sair de casa"), e o recebimento de notificações e alertas importantes (e.g., detecção de anomalias de consumo, alertas de segurança).
- **Painéis Físicos Dedicados:** Instalados em locais estratégicos da residência, como paredes de cômodos centrais, estes painéis podem variar de simples interruptores inteligentes programáveis a telas sensíveis ao toque sofisticadas. Proporcionam um ponto de controle centralizado e de acesso rápido para ajustes comuns, como iluminação, climatização e segurança, sem a necessidade de recorrer a um dispositivo móvel.
- **Assistentes Virtuais de Voz:** A integração com plataformas de assistência por voz (como Amazon Alexa, Google Assistant, Apple Siri, entre outras) permite o controle de dispositivos e a execução de tarefas por meio de comandos verbais. Esta forma de interação mãos-livres é particularmente conveniente para ações rápidas e adiciona uma camada de naturalidade e acessibilidade ao controle do sistema.

O design da interface deve priorizar a experiência do usuário (UX), focando na clareza da informação apresentada (e.g., gráficos de consumo compreensíveis, ícones representativos), na simplicidade da navegação entre diferentes funcionalidades e na capacidade de personalização para atender às preferências individuais dos moradores. Considerações de acessibilidade são igualmente importantes, garantindo que pessoas com diferentes habilidades, incluindo idosos ou indivíduos com deficiências visuais ou motoras, possam interagir com o sistema de forma eficaz. Com os esforços contínuos de padronização na indústria de automação residencial, a experiência do usuário tende a ser aprimorada, pois um único aplicativo ou assistente pode, idealmente, controlar dispositivos de diferentes marcas, simplificando a interação e reduzindo a complexidade para o usuário final. O objetivo final é abstrair a complexidade do sistema subjacente, fornecendo um meio de interação que seja ao mesmo tempo poderoso, flexível e fácil de usar.

2.1.6 Serviços em Nuvem e Análise de Dados

A integração com serviços em nuvem expande significativamente as capacidades dos sistemas de automação residencial, oferecendo recursos que transcendem o processamento e armazenamento local. A nuvem serve como uma plataforma escalável para:

- **Armazenamento Histórico de Dados em Larga Escala:** A vasta quantidade de dados gerada por sensores e dispositivos ao longo do tempo pode ser armazenada de forma eficiente e durável na nuvem, permitindo análises longitudinais e a manutenção de um histórico completo do comportamento do sistema e do consumo energético.
- **Análise Avançada de Dados:** Plataformas em nuvem frequentemente disponibilizam ferramentas sofisticadas para análise de big data e aprendizado de máquina. Isso permite a aplicação de algoritmos complexos para:
 - **Análise Preditiva:** Modelos podem ser treinados para antecipar consumos futuros com base em padrões históricos, condições climáticas previstas e rotinas dos moradores, permitindo uma otimização proativa do uso de energia.
 - **Deteção de Anomalias Sofisticada:** Além do CEP, algoritmos de aprendizado não supervisionado podem identificar padrões anormais sutis que indicariam falhas em equipamentos ou comportamentos de consumo atípicos.

- **Personalização e Recomendações:** Análise do comportamento individual dos usuários para oferecer recomendações personalizadas de economia de energia ou ajustes automáticos que melhorem o conforto.
- **Integração com Serviços Externos e APIs de Terceiros:** A nuvem facilita a conexão do sistema de automação residencial com uma miríade de outros serviços online. Exemplos incluem:
 - **Previsão do Tempo:** Ajustar automaticamente a climatização ou a irrigação com base nas condições meteorológicas futuras.
 - **Tarifas Dinâmicas de Energia:** Otimizar o consumo para aproveitar horários de tarifas mais baixas, caso a concessionária de energia ofereça tais planos.
 - **Serviços de Geolocalização:** Acionar cenas de "chegada em casa" ou "saída de casa" com base na localização dos smartphones dos moradores.
 - **Plataformas de Notificação:** Envio de alertas e relatórios para os usuários através de e-mail, SMS ou aplicativos de mensagens.
- **Acesso Remoto e Controle Global:** A nuvem é o facilitador primário para o controle e monitoramento do sistema residencial de qualquer lugar do mundo, através de aplicativos móveis ou interfaces web.
- **Manutenção Preditiva e Diagnóstico Remoto:** Fabricantes de dispositivos podem utilizar dados agregados (de forma anônima e com consentimento) para identificar falhas iminentes em equipamentos, possibilitando manutenções preventivas e atualizações de firmware remotas.
- **Backup e Recuperação de Desastres:** Serviços em nuvem oferecem soluções robustas para backup de configurações do sistema e dados históricos, garantindo a resiliência contra falhas de hardware local.

Embora a nuvem ofereça inúmeras vantagens, sua utilização também introduz considerações importantes sobre segurança, privacidade, dependência de conexão com a internet e custos de assinatura ou uso de serviços. Uma arquitetura bem planejada pode buscar um equilíbrio, utilizando a nuvem para funcionalidades que efetivamente se beneficiam de seus recursos, enquanto mantém operações críticas ou sensíveis à privacidade em execução local sempre que possível (computação de borda ou *edge computing*).

2.1.7 Segurança e Privacidade

À medida que os lares se tornam mais conectados e os sistemas de automação coletam um volume crescente de dados sobre os hábitos e rotinas dos moradores, as questões de segurança cibernética e privacidade de dados assumem uma importância primordial. A proteção contra acessos não autorizados, o uso indevido de informações pessoais e a garantia da conformidade com regulamentações como a Lei Geral de Proteção de Dados (LGPD) no Brasil e o Regulamento Geral sobre a Proteção de Dados (GDPR) na União Europeia são imperativos. Uma abordagem de *Privacy by Design* e *Security by Design* deve ser incorporada desde as fases iniciais de concepção do sistema, integrando mecanismos de proteção em todas as camadas da arquitetura.

As principais práticas e considerações de segurança incluem:

- **Criptografia Robusta:** Todos os dados sensíveis, tanto em trânsito (comunicação entre dispositivos, nuvem e aplicativos) quanto em repouso (armazenados em dispositivos, servidores locais ou na nuvem), devem ser protegidos por algoritmos de criptografia fortes e atualizados. Isso torna os dados ilegíveis para terceiros não autorizados, mesmo em caso de interceptação ou acesso indevido ao armazenamento, cumprindo os princípios de confidencialidade.
- **Autenticação e Autorização Fortes:** É crucial garantir que apenas dispositivos, usuários e serviços autenticados e autorizados possam acessar a rede doméstica e os dados do sistema. Isso envolve:
 - *Autenticação de Dispositivos:* Uso de certificados digitais, chaves criptográficas únicas ou outros métodos para verificar a identidade de cada dispositivo conectado.
 - *Autenticação de Usuários:* Implementação de senhas fortes, autenticação multifator (MFA) e gerenciamento adequado de credenciais de acesso às interfaces de usuário.
 - *Controle de Acesso Granular:* Definição de permissões específicas para diferentes usuários ou serviços, limitando o acesso apenas às funcionalidades e dados estritamente necessários para suas operações (princípio do menor privilégio).
- **Segurança de Rede:** Proteção da rede doméstica contra intrusões, utilizando firewalls, segmentação de rede (isolando dispositivos de automação em uma rede separada, se possível) e detecção de atividades suspeitas.

- **Atualizações Seguras e Gerenciamento de Vulnerabilidades:** Manter o firmware dos dispositivos, o software dos controladores e os aplicativos de interface sempre atualizados com as últimas correções de segurança é vital. O processo de atualização deve ser seguro, verificando a autenticidade e integridade do software para impedir a instalação de versões maliciosas. Um programa de monitoramento e correção de vulnerabilidades conhecidas deve estar em vigor.
- **Proteção contra Ataques Comuns:** Considerar defesas contra ataques como negação de serviço (DoS), *man-in-the-middle* (MitM), e exploração de interfaces de API inseguras.

A privacidade dos dados dos moradores é igualmente crítica. Informações sobre consumo de energia, horários de ocupação da residência, uso de dispositivos específicos e preferências pessoais podem revelar detalhes íntimos da vida cotidiana. As práticas para garantir a privacidade incluem:

- **Minimização da Coleta de Dados:** Coletar e reter apenas os dados estritamente necessários para a funcionalidade pretendida e pelo tempo indispensável (princípio da necessidade e da finalidade).
- **Anonimização e Pseudonimização:** Sempre que possível, os dados devem ser processados de forma anônima ou pseudônima para reduzir o risco de identificação dos indivíduos, especialmente para fins de análise estatística ou desenvolvimento de modelos.
- **Consentimento Informado e Transparência:** Os usuários devem ser claramente informados sobre quais dados são coletados, como são utilizados, com quem podem ser compartilhados e por quanto tempo são armazenados. O consentimento explícito deve ser obtido para o tratamento de dados pessoais, e os usuários devem ter controle sobre suas preferências de privacidade.
- **Direitos dos Titulares dos Dados:** Garantir que os usuários possam exercer seus direitos conforme previsto na LGPD/GDPR, como o direito de acesso, retificação, exclusão de seus dados e portabilidade.
- **Políticas de Privacidade Claras:** Disponibilizar políticas de privacidade de fácil compreensão que detalhem as práticas de tratamento de dados do sistema.
- **Auditoria e Responsabilização (*Accountability*):** Manter registros das operações de tratamento de dados e das medidas de segurança implementadas para demonstrar conformidade e facilitar investigações em caso de incidentes.

A confiança do usuário no sistema de automação residencial está intrinsecamente ligada à percepção de segurança e ao respeito à sua privacidade. Portanto, negligenciar esses aspectos pode comprometer não apenas a conformidade legal, mas também a aceitação e o sucesso da tecnologia.

2.1.8 Integração com Outros Sistemas

Um sistema de automação residencial raramente opera em completo isolamento. Seu verdadeiro potencial é frequentemente alcançado através da integração com uma variedade de outros sistemas e serviços, tanto internos quanto externos à residência. Essa integração permite a criação de um ecossistema doméstico mais coeso, inteligente e responsivo às necessidades dos moradores e às condições ambientais. As principais áreas de integração incluem:

- **Sistemas de Segurança Residencial:** A integração com alarmes, câmeras de vigilância, sensores de portas e janelas, e detectores de fumaça ou monóxido de carbono pode aprimorar significativamente a segurança. Por exemplo, em caso de detecção de intrusão, o sistema de automação pode acender todas as luzes, enviar notificações aos moradores e às autoridades, e até mesmo simular presença. Da mesma forma, dados de sensores de segurança podem informar decisões de automação, como desligar o sistema de climatização se uma janela for deixada aberta.
- **Sistemas de Entretenimento Doméstico:** A automação pode orquestrar sistemas de áudio e vídeo, criando ambientes imersivos. Cenas como "modo cinema" podem automaticamente diminuir as luzes, fechar cortinas e ligar o projetor e o sistema de som. A integração também pode permitir o controle de dispositivos de entretenimento através da mesma interface de automação.
- **Eletrodomésticos Inteligentes:** Um número crescente de eletrodomésticos (geladeiras, fornos, máquinas de lavar, etc.) vêm com conectividade e APIs. A integração permite que estes sejam incluídos em rotinas de automação, monitorados quanto ao consumo energético, e controlados remotamente. Por exemplo, uma máquina de lavar louça pode ser programada para operar em horários de tarifa de energia mais baixa.
- **Sistemas de Geração Distribuída de Energia e Armazenamento:** Para residências com painéis solares fotovoltaicos ou sistemas de armazenamento de energia (baterias), a integração é crucial. O sistema de automação pode otimizar

o autoconsumo da energia gerada, decidir quando carregar ou descarregar baterias com base na demanda, na tarifa de energia e na previsão de geração solar, e até mesmo participar de programas de resposta à demanda da rede elétrica.

- **Sistemas de Gerenciamento Predial (BMS):** Em edifícios multifamiliares ou condomínios, a integração com sistemas de gerenciamento predial pode permitir o controle coordenado de recursos compartilhados e a otimização energética em uma escala maior.
- **Plataformas de Terceiros e Serviços Online:** Conforme mencionado na seção sobre serviços em nuvem, a integração via APIs com serviços de previsão do tempo, calendários, assistentes de voz, e plataformas de e-commerce pode enriquecer as funcionalidades do sistema, permitindo automações mais contextuais e personalizadas.

No entanto, a integração de múltiplos sistemas e plataformas introduz desafios significativos. A complexidade na gestão dos dados aumenta, exigindo mapeamento cuidadoso de informações e protocolos. A interoperabilidade entre diferentes padrões e tecnologias continua sendo uma barreira, embora esforços de padronização visem mitigar esse problema. Fundamentalmente, ao integrar sistemas, é crucial garantir que as políticas de segurança e privacidade sejam mantidas de forma coesa em todo o ecossistema. Cada sistema conectado representa um potencial vetor de ataque ou uma fonte de vazamento de dados, exigindo uma avaliação de risco cuidadosa e a implementação de medidas de proteção adequadas, incluindo a verificação da conformidade de serviços externos com normas como a LGPD. A gestão de identidades e o controle de acesso entre sistemas integrados também se tornam mais complexos e necessitam de atenção especial.

2.1.9 Escalabilidade e Flexibilidade

A capacidade de um sistema de automação residencial crescer e se adaptar às necessidades mutáveis dos moradores, bem como à evolução tecnológica, é um indicador crucial de sua longevidade e valor a longo prazo. Escalabilidade refere-se à habilidade do sistema de lidar com um número crescente de dispositivos, sensores, usuários e volume de dados sem degradação significativa de desempenho. Flexibilidade, por sua vez, denota a facilidade com que o sistema pode ser modificado, atualizado ou reconfigurado para acomodar novas funcionalidades, tecnologias ou cenários de uso.

Diversos fatores contribuem para a escalabilidade e flexibilidade de um sistema:

- **Arquitetura Modular:** Projetar o sistema em módulos independentes e bem definidos (e.g., camada de aquisição, processamento, controle, interface) permite que cada componente seja atualizado ou substituído com impacto mínimo sobre os demais. Interfaces de Programação de Aplicativos (APIs) bem documentadas entre os módulos são essenciais para essa abordagem.
- **Uso de Padrões Abertos:** Adoção de protocolos de comunicação, formatos de dados e APIs padronizados e abertos facilita a integração de dispositivos e serviços de diferentes fabricantes, evitando o aprisionamento tecnológico (*vendor lock-in*) e promovendo um ecossistema mais competitivo e inovador. Isso permite que os usuários escolham entre uma gama maior de produtos e que o sistema possa incorporar novas tecnologias à medida que surgem.
- **Design Orientado a Serviços:** Em arquiteturas mais complexas, especialmente aquelas que envolvem componentes em nuvem, uma abordagem orientada a microsserviços pode oferecer alta escalabilidade, permitindo que serviços individuais sejam dimensionados independentemente conforme a demanda.
- **Capacidade de Descoberta de Dispositivos:** Mecanismos que permitem ao sistema descobrir e configurar automaticamente novos dispositivos adicionados à rede simplificam a expansão e melhoram a experiência do usuário.
- **Configurabilidade e Personalização:** O sistema deve permitir que os usuários (ou administradores, em cenários mais complexos) configurem facilmente regras de automação, personalizem interfaces e ajustem parâmetros operacionais para atender às suas preferências específicas.
- **Suporte a Atualizações de Software e Firmware:** A capacidade de atualizar remotamente o software dos controladores, gateways e o firmware dos dispositivos é crucial para adicionar novas funcionalidades, corrigir bugs e aplicar patches de segurança ao longo do ciclo de vida do sistema.
- **Planejamento de Capacidade:** Para sistemas que lidam com grandes volumes de dados (especialmente dados históricos de monitoramento energético), é importante considerar a escalabilidade da infraestrutura de armazenamento e processamento, seja ela local ou baseada em nuvem.

A escalabilidade e a flexibilidade não são apenas atributos técnicos; elas impactam diretamente a usabilidade e o custo total de propriedade do sistema. Um sistema que pode ser facilmente expandido para incluir um novo cômodo, integrar um novo tipo de sensor ou adaptar-se a novas rotinas familiares oferece um valor contínuo. Por outro

lado, um sistema rígido e difícil de modificar pode rapidamente se tornar obsoleto ou exigir substituições dispendiosas.

Neste contexto, é fundamental que qualquer expansão ou modificação do sistema mantenha o mesmo nível de rigor em relação à segurança e privacidade. A introdução de novos dispositivos, fluxos de dados ou funcionalidades deve ser acompanhada de uma reavaliação dos riscos e da aplicação das devidas medidas de proteção, garantindo a conformidade contínua com regulamentações como a LGPD e a proteção das informações pessoais dos moradores.

2.1.10 Instrumentação do Processo

A instrumentação do processo em um sistema de automação residencial refere-se à cuidadosa seleção, instalação física, configuração lógica e comissionamento de todos os componentes de hardware, incluindo sensores, atuadores e controladores. Uma instrumentação bem executada é fundamental não apenas para a funcionalidade e confiabilidade do sistema, mas também para garantir a qualidade dos dados coletados, a eficácia das ações de controle e a segurança operacional e informacional. Além disso, deve assegurar que o tratamento das informações coletadas esteja em conformidade com os princípios da LGPD e outras normativas de proteção de dados.

Os principais aspectos a serem considerados durante a instrumentação incluem:

- **Seleção Criteriosa de Hardware:** A escolha dos componentes deve ir além das especificações técnicas básicas (como faixa de operação, precisão, e compatibilidade). É crucial considerar:
 - *Confiabilidade e Durabilidade:* Optar por dispositivos de fabricantes com boa reputação e histórico de produtos robustos.
 - *Segurança Embarcada:* Verificar se o hardware suporta funcionalidades de segurança essenciais, como armazenamento seguro de chaves criptográficas, inicialização segura (*secure boot*), e se o fabricante fornece atualizações de firmware para correção de vulnerabilidades.
 - *Suporte a Padrões Abertos:* Dispositivos que aderem a padrões de comunicação e interoperabilidade abertos facilitam a integração e a evolução futura do sistema.
 - *Compromisso do Fabricante com Privacidade:* Avaliar as políticas de privacidade do fabricante e seu histórico em relação à proteção de dados do usuário, especialmente para dispositivos que se conectam a serviços em nuvem.

- *Consumo Energético do Próprio Dispositivo*: Para sensores e atuadores alimentados por bateria, o baixo consumo é essencial. Mesmo para dispositivos alimentados pela rede, a eficiência energética contribui para a sustentabilidade geral do sistema.

- **Planejamento da Instalação Física e Topologia da Rede:**

- *Posicionamento Estratégico de Sensores*: A localização dos sensores (e.g., de temperatura, presença, luminosidade) deve ser otimizada para garantir medições representativas e evitar interferências ambientais que possam distorcer os dados.
- *Cobertura da Rede de Comunicação*: Para sistemas sem fio, é necessário planejar a disposição dos dispositivos e, se necessário, de repetidores ou extensores de sinal para garantir uma cobertura robusta e confiável em todas as áreas desejadas. Em redes *mesh*, a densidade dos nós deve ser suficiente para assegurar redundância e múltiplos caminhos de comunicação.
- *Minimização de Interferências*: Considerar fontes potenciais de interferência eletromagnética (para dispositivos sem fio) ou ruído elétrico (para comunicação via *powerline*) e adotar medidas para mitigá-las.
- *Segmentação da Rede*: Por razões de segurança, pode ser vantajoso isolar a rede de dispositivos de automação da rede principal utilizada para computadores e dispositivos pessoais, limitando a superfície de ataque em caso de comprometimento de um dispositivo.

- **Integração Elétrica Segura e Confiável**: A instalação de atuadores, controladores e sensores alimentados pela rede elétrica deve seguir rigorosamente as normas técnicas e de segurança elétrica. Isso inclui o uso de fiação adequada, disjuntores de proteção, aterramento correto e isolamento apropriado para prevenir curtos-circuitos, sobrecargas ou choques elétricos, que podem não apenas danificar os equipamentos, mas também comprometer a segurança dos moradores e a integridade dos dados.

- **Provisionamento e Configuração Segura de Dispositivos**: O processo de adicionar novos dispositivos ao sistema (provisionamento) deve ser seguro para evitar que dispositivos não autorizados sejam incorporados à rede. Isso envolve:

- *Configuração Inicial Segura*: Alteração de senhas padrão, configuração de chaves de rede seguras e únicas.

- *Autenticação Mútua*: Idealmente, tanto o dispositivo quanto o controlador/gateway devem autenticar um ao outro antes de estabelecer a comunicação.
 - *Gerenciamento de Chaves Criptográficas*: Implementação de práticas seguras para a geração, distribuição e armazenamento de chaves criptográficas utilizadas para proteger a comunicação.
 - *Rotinas de Atualização de Firmware*: Estabelecer um processo para manter o firmware dos dispositivos atualizado, garantindo que as atualizações sejam obtidas de fontes confiáveis e instaladas de forma segura.
- **Calibração e Testes**: Após a instalação, sensores e atuadores podem necessitar de calibração para garantir a precisão das medições e a correta execução das ações. Testes exaustivos de todos os componentes e funcionalidades do sistema são cruciais para verificar a correta instrumentação e identificar quaisquer problemas antes da operação plena.

Em suma, a instrumentação do processo é uma etapa multidisciplinar que combina conhecimentos de engenharia elétrica, redes de comunicação, segurança da informação e, cada vez mais, conformidade regulatória. Um cuidado meticuloso nesta fase é o alicerce para um sistema de automação residencial que não apenas funcione de maneira eficiente e confiável, mas que também proteja os dados e a privacidade de seus usuários, operando em total conformidade com as boas práticas e as leis vigentes.

2.2 Monitoramento Energético

O monitoramento energético é um elemento central para a compreensão aprofundada do uso de recursos em um ambiente residencial automatizado. Mais do que simplesmente quantificar o consumo de eletricidade, o monitoramento energético permite revelar padrões, prever demandas futuras, identificar ineficiências e propor intervenções corretivas ou preventivas. Ao integrar dados provenientes de diversos sensores — incluindo aqueles dedicados à medição de corrente e tensão em circuitos específicos, bem como às tomadas e equipamentos inteligentes —, o sistema obtém uma visão holística do comportamento energético da residência.

Abaixo, são detalhadas as principais etapas e aspectos relacionados ao monitoramento energético, com especial ênfase na análise de dados, privacidade, conformidade com a LGPD, aplicações práticas e o uso de técnicas de controle estatístico de processo.

2.2.1 Coleta e Organização dos Dados

A primeira etapa do monitoramento energético consiste na coleta sistemática de dados relativos ao consumo de eletricidade em diferentes pontos da residência. Esses dados podem ser obtidos por meio de:

- **Sensores de Consumo em Tempo Real:** Dispositivos que medem corrente e tensão, permitindo o cálculo instantâneo da potência e energia consumidas em circuitos específicos ou em eletrodomésticos individuais.
- **Tomadas e Disjuntores Inteligentes:** Elementos capazes de registrar o uso energético de um aparelho conectado, bem como ligar ou desligar o fornecimento conforme instruções do controlador.
- **Medição Global da Residência:** Sensores posicionados no quadro elétrico principal, fornecendo uma visão macro do consumo, contra a qual se podem comparar dados mais granulares para identificar fontes de ineficiência.

A coleta contínua gera um grande volume de dados. Para lidar com isso, é essencial adotar uma infraestrutura de armazenamento e processamento robusta. As soluções de armazenamento podem variar desde dispositivos de armazenamento conectados à rede local (*Network Attached Storage* - NAS), que oferecem um repositório centralizado e privado, até sistemas baseados em nuvem ou servidores locais dedicados. Cada abordagem apresenta suas próprias vantagens em termos de capacidade, escalabilidade, custo, segurança e facilidade de acesso. A escolha dependerá dos requisitos específicos do sistema, do volume de dados esperado e das políticas de privacidade e segurança adotadas. Independentemente da solução, é crucial garantir mecanismos de acesso controlado, backups regulares e capacidade de expansão. Além disso, a utilização de sistemas híbridos, combinando armazenamento local com recursos em nuvem, pode oferecer um equilíbrio entre resiliência, segurança e acessibilidade dos dados.

Em qualquer cenário, devem-se implementar mecanismos de segurança (criptografia, autenticação forte) e técnicas de compressão e normalização de dados para otimizar o armazenamento e facilitar o processamento posterior.

2.2.2 Pré-Processamento, Qualidade dos Dados e Controle Estatístico de Processo

Antes de realizar análises avançadas, os dados coletados precisam passar por etapas de pré-processamento, incluindo:

- **Limpeza de Dados:** Remoção de leituras espúrias, tratamento de valores faltantes e correção de discrepâncias causadas por ruído elétrico ou falhas de comunicação.
- **Agregação e Granularidade:** Ajuste da escala temporal (por exemplo, sumarizar leituras segundo a segundo, minuto a minuto ou hora a hora) e espacial (agrupando dados por cômodo, circuito ou categoria de aparelho).
- **Normalização e Padronização:** Transformação dos dados em escalas comparáveis, permitindo análises consistentes entre diferentes pontos de medição ou períodos.

Um conjunto de dados bem pré-processado é mais coerente, confiável e adequado à aplicação de técnicas de análise estatística, aprendizado de máquina e modelagem preditiva.

Nesse contexto, o **Controle Estatístico de Processo (CEP)** pode ser aplicado ao monitoramento energético como uma ferramenta para acompanhar a variabilidade do consumo ao longo do tempo. Oriundo do domínio da qualidade industrial, o CEP utiliza gráficos de controle e limites estatísticos para distinguir variações normais de anomalias. Aplicado à automação residencial, o CEP permite:

- **Estabelecer Faixas Normais de Operação:** Determinar limites superiores e inferiores para o consumo esperado, considerando padrões históricos e características da residência.
- **Deteção Precoce de Anomalias:** Identificar rapidamente desvios significativos, como um aumento inesperado no consumo de um aparelho, indicando falha ou uso indevido.
- **Monitoramento Contínuo:** Acompanhar a evolução do consumo ao longo do tempo, avaliando se as mudanças implementadas (como troca de um equipamento ou ajuste nas rotinas) resultam em maior eficiência.

O CEP atua como um complemento às técnicas de análise mais complexas, ajudando a manter o sistema dentro de parâmetros normais e fornecendo bases para intervenções pontuais.

2.2.3 Análise Estatística e Inteligência Artificial

Com dados limpos e estruturados, bem como limites estatísticos definidos pelo CEP, torna-se possível aplicar uma variedade de técnicas analíticas para extrair valor significativo:

- **Estatística Descritiva:** Cálculo de médias, medianas, desvios-padrão e percentis para caracterizar o perfil de consumo energético ao longo do tempo.
- **Deteção de Anomalias Avançada:** Em conjunto com o CEP, métodos de aprendizado de máquina podem reconhecer padrões não lineares, detectando anomalias sutis que escapariam a métodos estatísticos convencionais.
- **Análise de Correlação e Causalidade:** Estabelecer relações entre o consumo energético e outras variáveis (temperatura, ocupação, eventos externos).
- **Modelagem Preditiva:** Uso de algoritmos de aprendizado supervisionado ou não supervisionado para prever demandas futuras, ajustar parâmetros de controle em tempo real e recomendar intervenções.
- **Clusterização de Perfis de Consumo:** Agrupamento de padrões de uso semelhantes, auxiliando na identificação de zonas energéticas e na otimização personalizada do sistema.

A inteligência artificial, combinada ao CEP, amplia as capacidades do sistema, tornando-o mais adaptativo e robusto. Isso resulta em ajustes proativos na operação da residência, reduzindo custos, melhorando o conforto e minimizando impactos ambientais.

2.2.4 Privacidade, LGPD e Proteção dos Dados Energéticos

As informações sobre consumo energético podem revelar hábitos, presença, rotinas e preferências dos moradores. Como tais dados podem ser considerados pessoais ou sensíveis segundo a LGPD, devem ser tratados com o mesmo rigor aplicado a outras informações coletadas:

- **Minimização de Dados:** Coletar apenas o nível de detalhe necessário.
- **Anonimização e Pseudonimização:** Remover ou mascarar informações identificadoras.
- **Consentimento e Transparência:** Explicar claramente aos moradores como, por que e por quanto tempo os dados serão coletados, analisados e armazenados.
- **Segurança da Informação:** Aplicar criptografia ponta a ponta, autenticação robusta e isolamento de redes.
- **Auditorias e Responsabilização:** Manter registros sobre o tratamento de dados, comprovando conformidade com a LGPD em caso de inspeções.

Ao equilibrar análise de dados avançada, CEP e privacidade, o monitoramento energético torna-se não apenas um recurso valioso, mas também uma solução respeitosa aos direitos e expectativas dos usuários.

2.2.5 Integração com Outros Sistemas e Evolução Contínua

O monitoramento energético não existe isoladamente. Ao integrar-se com controladores, sistemas de climatização, iluminação e segurança, bem como com serviços em nuvem e APIs externas, o sistema pode ajustar em tempo real o uso de energia conforme necessidades, demandas e condições externas.

Com a aplicação do CEP, é possível identificar se as integrações e ajustes recomendados estão mantendo o consumo dentro dos limites estatísticos esperados ou se intervenções adicionais são necessárias. Conforme novos dispositivos e tecnologias surgem, a arquitetura flexível e o uso de padrões abertos asseguram que o sistema seja escalável e possa incorporar inovações sem comprometer a segurança, a privacidade ou a qualidade da análise de dados.

2.2.6 Benefícios Práticos e Responsabilidade Social

Quando bem implementado, o monitoramento energético, aliado ao CEP e à análise de dados, traz diversos benefícios:

- **Eficiência e Economia:** Redução de custos com energia elétrica por meio da identificação de desperdícios e ajustes estratégicos.
- **Conforto e Qualidade de Vida:** Ajustes automáticos de parâmetros ambientais conforme preferências e necessidades, sem intervenção manual constante.
- **Sustentabilidade:** Otimização do uso de energia, priorizando fontes renováveis e reduzindo a pegada de carbono.
- **Tomada de Decisão Baseada em Evidências:** Dados claros, validados e dentro de parâmetros estatísticos auxiliares guiam melhorias contínuas e embasadas em fatos.

A responsabilidade social emerge ao assegurar que tais benefícios sejam obtidos sem invadir a privacidade ou violar direitos fundamentais. A conformidade com a LGPD, o uso ético da análise de dados e a adoção de CEP garantem que a tecnologia melhore a vida dos moradores, mantendo a confiança e a proteção das informações pessoais. Em síntese, o monitoramento energético aplicado a um sistema de automação residencial,

quando embasado em um arcabouço analítico sólido, conformidade regulatória, uso de CEP e cuidado com a privacidade, transcende a mera coleta de dados. Ele estabelece um processo contínuo de aprendizado, ajuste e otimização que beneficia usuários, meio ambiente e sociedade em geral.

2.3 Modelos Arquiteturais para Sistemas de Automação Residencial e Monitoramento Energético

Com base nos princípios discutidos anteriormente, um sistema integrado para monitoramento energético e automação residencial tipicamente adota uma arquitetura em camadas. Esta abordagem modular visa oferecer uma solução prática e inteligente, especialmente para residências sem infraestrutura de automação preexistente, permitindo flexibilidade e escalabilidade. A arquitetura pode ser conceitualizada em torno de três pilares principais: a camada de aquisição de dados, a camada de processamento e gerenciamento de dados, e a camada de apresentação e interação com o usuário, complementada por uma lógica de automação e agendamento.

2.3.1 Visão Geral de um Sistema Típico

Um sistema de automação residencial com foco em monitoramento energético opera em um fluxo contínuo. Este fluxo inicia-se com a aquisição de dados de consumo e estado dos dispositivos em tempo real. Estes dados brutos são então transmitidos para uma camada de processamento, onde são limpos, transformados, enriquecidos e armazenados de forma estruturada. Finalmente, esta informação processada é consumida pela camada de apresentação, que oferece ao usuário final uma interface para visualização de insights, análises e funcionalidades de controle direto sobre os dispositivos, bem como para a configuração de rotinas de automação.

2.3.2 Camada de Aquisição de Dados

A base da coleta de dados do sistema reside em dispositivos inteligentes capazes de medir e reportar informações relevantes. No contexto do monitoramento energético, estes incluem:

- **Dispositivos de Medição Dedicados:** Tomadas inteligentes, disjuntores inteligentes, ou medidores de energia em linha que monitoram parâmetros como tensão (V), corrente (A), potência ativa (W) e consumo acumulado de energia (kWh) de aparelhos específicos ou circuitos inteiros.

- **Sensores Ambientais e de Contexto:** Sensores de temperatura, luminosidade, ocupação, entre outros, que fornecem dados contextuais que podem ser correlacionados com o consumo de energia.

Estes dispositivos geralmente se conectam a uma rede local (e.g., Wi-Fi, Zigbee, Z-Wave) e podem transmitir seus dados para um gateway local ou diretamente para uma plataforma em nuvem fornecida pelo fabricante ou por um integrador de sistemas. A escolha dos dispositivos e da forma de comunicação depende de fatores como custo, facilidade de instalação, precisão da medição e capacidade de integração com o restante da arquitetura. A coleta de dados deve ser periódica e com granularidade suficiente para permitir análises significativas.

2.3.3 Camada de Processamento e Gerenciamento de Dados

Para garantir que os dados coletados sejam transformados em informações úteis e armazenados de forma confiável e automatizada, é necessária uma robusta pipeline de processamento de dados. Esta camada é responsável por:

- **Extração de Dados (Extract):** Coleta dos dados brutos das fontes (gateways locais, APIs de nuvem de dispositivos).
- **Transformação de Dados (Transform):**
 - **Limpeza:** Remoção de ruídos, tratamento de dados faltantes ou inconsistentes.
 - **Conversão e Normalização:** Ajuste de unidades, conversão de timestamps para formatos padronizados.
 - **Enriquecimento:** Adição de metadados, como nomes amigáveis para dispositivos (a partir de um mapeamento), localização, ou informações contextuais.
 - **Agregação:** Sumarização dos dados em diferentes granularidades temporais (e.g., por minuto, hora, dia) ou espaciais (e.g., por cômodo).
- **Carga de Dados (Load):** Armazenamento dos dados processados e enriquecidos em um repositório adequado. Formatos colunares (como Parquet ou ORC) são frequentemente escolhidos para cargas de trabalho analíticas devido à sua eficiência em armazenamento e consulta. O armazenamento pode ser particionado (e.g., por data, por dispositivo) para otimizar as consultas.

2.3. *MODELOS ARQUITETURAIS PARA SISTEMAS DE AUTOMAÇÃO RESIDENCIAL E MONITORAMENTO*

Ferramentas de orquestração de fluxos de trabalho são comumente utilizadas para automatizar, agendar e monitorar essas pipelines de dados. Elas gerenciam dependências entre as etapas de processamento, lidam com falhas e fornecem visibilidade sobre a execução da pipeline, facilitando a manutenção e a evolução do processo de ETL (Extract, Transform, Load) ou ELT (Extract, Load, Transform) dos dados energéticos.

2.3.4 Camada de Apresentação e Interação com o Usuário

A interação do usuário com o sistema é realizada através de uma interface, que pode assumir diversas formas:

- **Aplicações Web ou Móveis:** Permitem aos usuários visualizar dados históricos e em tempo real, configurar dispositivos, criar cenas de automação e receber notificações. Dashboards com gráficos de consumo, métricas de eficiência e comparativos são comuns.
- **Painéis de Controle Dedicados:** Dispositivos físicos, muitas vezes com telas sensíveis ao toque, instalados na residência para acesso rápido a controles e informações.
- **Assistentes de Voz:** Integração com plataformas de assistentes virtuais permite o controle de dispositivos e a consulta de informações por meio de comandos de voz.

Esta camada deve ser projetada com foco na experiência do usuário (UX), oferecendo uma navegação intuitiva, informações claras e controle responsivo. As funcionalidades típicas incluem:

- **Visualização de Consumo:** Apresentação de dados de consumo energético de forma compreensível (e.g., consumo total, por dispositivo, tendências ao longo do tempo).
- **Análises Detalhadas:** Métricas de qualidade de energia (tensão, corrente), perfis de consumo horário, comparativos entre períodos.
- **Controle de Dispositivos:** Capacidade de ligar/desligar ou ajustar dispositivos remotamente.
- **Gerenciamento de Cenas e Rotinas:** Interface para criar, modificar e excluir regras de automação baseadas em horários, eventos de sensores ou comandos do usuário.

A modularidade na concepção desta camada, utilizando, por exemplo, componentes de UI reutilizáveis e APIs bem definidas para acesso aos dados processados, facilita sua manutenção e evolução.

2.3.5 Lógica de Automação e Agendamento de Rotinas

Paralelamente à interação direta do usuário, um componente crucial do sistema é o motor de automação, responsável por executar as regras e cenas definidas. Esta lógica pode residir em controladores locais, em gateways ou em serviços de backend.

- **Definição de Regras:** Os usuários (ou o próprio sistema, através de aprendizado) definem regras do tipo "se-então" (e.g., "se o sensor de presença não detectar movimento por 10 minutos, apagar as luzes do cômodo").
- **Cenas (Scenarios/Routines):** Conjuntos pré-definidos de ações sobre múltiplos dispositivos que podem ser ativados por um único comando, horário ou evento (e.g., cena "Sair de Casa" que desliga luzes, ajusta o termostato e ativa o sistema de segurança).
- **Agendamento:** Execução de ações ou cenas em horários específicos e com recorrência definida (e.g., ligar a irrigação do jardim às 6h da manhã às segundas, quartas e sextas).

Um serviço de agendamento robusto, muitas vezes executado no backend, monitora continuamente as condições e os horários para disparar as automações programadas. Este serviço interage com a camada de controle dos dispositivos para enviar os comandos necessários. A separação desta lógica da interface do usuário garante que as automações funcionem de forma independente, mesmo que a aplicação de interface não esteja ativa.

A integração coesa dessas camadas — aquisição, processamento, apresentação e automação — permite a criação de sistemas de automação residencial e monitoramento energético que são não apenas funcionais, mas também inteligentes, adaptáveis e centrados no usuário.

Capítulo 3

Metodologia

Neste capítulo, são detalhadas as etapas metodológicas adotadas no desenvolvimento do sistema inteligente para monitoramento e controle estatístico de processo (CEP) para o monitoramento de energia residencial. A abordagem foi projetada para atender aos objetivos do projeto, combinando análise técnica, desenvolvimento de soluções e validação experimental. Os procedimentos descritos têm como objetivo garantir a reprodução, escalabilidade e eficácia do sistema proposto, contribuindo para o avanço na área de automação e eficiência energética.

3.1 Pesquisa e Análise de Tecnologias Existentes

A fase inicial do projeto, crucial para a definição da arquitetura do sistema, consistiu na identificação e análise aprofundada das diversas tecnologias e abordagens disponíveis para o monitoramento de consumo energético em ambientes residenciais. Esta etapa investigativa envolveu a consulta a artigos científicos, documentação técnica de fabricantes, white papers, fóruns de discussão de desenvolvedores e a análise de produtos disponíveis no mercado consumidor. Realizou-se um estudo comparativo de soluções, avaliando alternativas que variavam desde o desenvolvimento de hardware customizado até a utilização de dispositivos comerciais com diferentes protocolos de comunicação. O objetivo primordial era encontrar um equilíbrio ótimo entre capacidade técnica, esforço de desenvolvimento, custo de implementação e alinhamento com os objetivos centrais do PFC. A seleção da tecnologia mais adequada foi, portanto, pautada por um conjunto de critérios técnicos e práticos, detalhados a seguir.

Diversas abordagens para medição e coleta de dados foram consideradas:

1. **Desenvolvimento de Hardware Customizado:** Esta abordagem, embora considerada, implicaria no projeto e construção de um dispositivo de medição de

energia totalmente novo. A concepção típica envolveria a seleção de um microcontrolador adequado, como um ESP32 (conhecido por sua conectividade Wi-Fi e Bluetooth integradas e capacidade de processamento) ou um Arduino (popular pela sua simplicidade e vasta comunidade de suporte), que seria acoplado a sensores de corrente não invasivos (transformadores de corrente como o SCT-013, que medem a corrente sem interromper o circuito) e sensores de tensão (geralmente divisores de tensão resistivos ou transformadores de potencial para reduzir a tensão da rede a níveis seguros para o microcontrolador). Seria necessário também projetar um circuito de condicionamento de sinal para garantir que as leituras dos sensores fossem precisas e estáveis, além de uma fonte de alimentação para o dispositivo.

- *Vantagens:* A principal vantagem reside na máxima flexibilidade. O desenvolvedor teria controle total sobre o design do hardware, a escolha dos componentes (permitindo otimizar para precisão, custo ou tamanho), a frequência de amostragem dos dados (podendo ser ajustada para capturar transientes rápidos ou economizar energia), e o formato dos dados transmitidos. Esta rota oferece um profundo aprendizado sobre os princípios de medição de energia, eletrônica de potência e design de sistemas embarcados. A comunicação dos dados poderia ser customizada, utilizando Wi-Fi para conexão direta à rede local, Ethernet para conexões cabeadas mais estáveis, ou outros módulos de rádio frequência (como LoRa ou NB-IoT) para cenários específicos de longo alcance ou baixo consumo.
- *Desvantagens:* Esta opção apresenta desafios significativos. Requer conhecimento especializado e multidisciplinar em eletrônica analógica e digital, design de PCB (Printed Circuit Board) para criar uma placa compacta e confiável, técnicas de calibração de sensores para garantir a acurácia das medições, e programação embarcada (geralmente em C/C++ ou MicroPython) para ler os sensores, processar os dados e implementar a comunicação. Os custos de desenvolvimento e prototipagem, mesmo para uma única unidade, podem ser significativos, considerando a aquisição de componentes, ferramentas de desenvolvimento (como osciloscópios e multímetros de precisão) e múltiplas iterações de PCB. O tempo de desenvolvimento é consideravelmente maior em comparação com a utilização de soluções comerciais prontas. Além disso, a segurança elétrica é uma preocupação primordial ao lidar diretamente com tensões de rede (127V/220V), exigindo um design cuidadoso para evitar riscos de choque elétrico ou incêndio.

2. Tomadas Inteligentes Comerciais com Protocolos Diversos: Uma alternativa mais pragmática e alinhada com o foco do projeto em software e integrações é a utilização de tomadas inteligentes (smart plugs) disponíveis comercialmente. Estes dispositivos já incorporam funcionalidades de medição de energia (potência, corrente, tensão e, por vezes, consumo acumulado) em um formato compacto e de fácil instalação. O mercado oferece uma vasta gama desses dispositivos, que se diferenciam principalmente pelo protocolo de comunicação que utilizam para se conectar a uma rede doméstica ou a um sistema de controle central. A escolha do protocolo impacta diretamente a necessidade de hardware adicional (como hubs), a complexidade da configuração, o alcance da comunicação e, crucialmente para este projeto, as opções de acesso aos dados via API.

- *Protocolo Wi-Fi (como a linha Tuya):* Dispositivos baseados em Wi-Fi, como os da popular plataforma Tuya, conectam-se diretamente ao roteador Wi-Fi doméstico existente (tipicamente 2.4GHz), eliminando a necessidade de um hub ou gateway central. A configuração é geralmente realizada de forma simples através de um aplicativo móvel fornecido pelo fabricante, onde o usuário associa o dispositivo à sua rede Wi-Fi e à sua conta na nuvem do fabricante. A comunicação com a nuvem é uma característica comum, permitindo o controle remoto e o acesso aos dados de qualquer lugar com conexão à internet. A disponibilidade e a qualidade de uma API de nuvem para acesso programático aos dados de medição e para envio de comandos de controle (ligar/desligar) são fatores decisivos ao considerar esta categoria.
- *Protocolos de Baixo Consumo e Redes Mesh (e.g., Zigbee, Z-Wave):* Protocolos como Zigbee e Z-Wave são especificamente projetados para automação residencial e Internet das Coisas (IoT). Eles operam em frequências de rádio diferentes do Wi-Fi (e.g., 908.42 MHz para Z-Wave na América do Norte, 2.4 GHz para Zigbee globalmente, mas fora das bandas Wi-Fi congestionadas), e são otimizados para baixo consumo de energia e para a criação de redes mesh robustas. Em uma rede mesh, os dispositivos podem retransmitir sinais uns dos outros, aumentando o alcance e a confiabilidade da rede, especialmente em residências maiores ou com muitas obstruções. No entanto, dispositivos Zigbee ou Z-Wave normalmente requerem um hub ou gateway específico do protocolo para atuar como uma ponte entre a rede mesh de baixo consumo e a rede local (LAN) via Wi-Fi ou Ethernet e, subseqüentemente, à internet. Este hub centraliza a comunicação e pode oferecer uma API local ou na nuvem para integração com outros sistemas. A necessidade

do hub adiciona um componente de custo e configuração ao sistema.

- *Bluetooth Low Energy (BLE)*: Dispositivos que utilizam Bluetooth Low Energy (BLE) são caracterizados pelo consumo energético extremamente baixo, permitindo que operem por longos períodos com baterias pequenas. Embora existam alguns dispositivos de monitoramento que utilizam BLE, seu uso para monitoramento contínuo de energia em tomadas (que são alimentadas pela rede elétrica) é menos comum em comparação com Wi-Fi, Zigbee ou Z-Wave para este tipo de aplicação. Para um sistema de monitoramento residencial completo e contínuo, dispositivos BLE geralmente necessitariam de um gateway BLE (um dispositivo sempre ativo, como um smartphone dedicado, um computador ou um hub específico) para coletar os dados dos diversos sensores BLE e retransmiti-los via Wi-Fi ou Ethernet para uma plataforma de análise ou armazenamento. O alcance do BLE é relativamente limitado (tipicamente até 10-30 metros em ambientes internos, dependendo das obstruções), e a conexão direta contínua com um smartphone para coleta de dados de múltiplos dispositivos não é uma solução prática ou escalável para monitoramento residencial 24/7.

A avaliação dessas alternativas foi realizada com base nos seguintes critérios fundamentais para os objetivos do projeto:

- **Compatibilidade com infraestruturas residenciais convencionais**: Este critério prioriza tecnologias que minimizam a necessidade de modificações estruturais ou elétricas complexas na residência. Soluções *plug and play*, como tomadas inteligentes, são altamente desejáveis, pois podem ser facilmente instaladas por usuários finais sem conhecimento técnico. O desenvolvimento de hardware customizado, embora pudesse ser projetado para ser não invasivo (e.g., com sensores de corrente de encaixe), ainda apresentaria uma barreira de instalação e segurança maior. Dispositivos Zigbee/Z-Wave, embora também sejam frequentemente *plug and play*, introduzem a necessidade de um hub, adicionando um componente à infraestrutura.
- **Eficiência técnica**: Refere-se à precisão, resolução e confiabilidade das medições de energia (potência, corrente, tensão, consumo acumulado), bem como à robustez do dispositivo e à frequência de coleta de dados. Hardware customizado permitiria, teoricamente, alta precisão, mas exigiria calibração cuidadosa. Dispositivos comerciais variam em qualidade; a documentação técnica sobre a precisão dos sensores internos nem sempre é transparente. A capacidade de obter dados

com granularidade suficiente (e.g., a cada poucos segundos ou minutos) é vital para análises como o CEP.

- **Interoperabilidade:** Avalia a capacidade da tecnologia de se integrar com outros sistemas, plataformas ou protocolos. Padrões abertos como Zigbee e Z-Wave são projetados com a interoperabilidade em mente, embora muitas vezes dentro de seus próprios ecossistemas ou através de plataformas de automação que suportam múltiplos protocolos. Dispositivos Wi-Fi podem ser mais isolados, dependendo da abertura da API do fabricante. Hardware customizado poderia ser projetado para usar protocolos abertos (e.g., MQTT), mas isso adiciona complexidade ao desenvolvimento.
- **Custo-benefício:** Considera o custo total da solução, incluindo aquisição de hardware, desenvolvimento (se aplicável), instalação e manutenção, em relação aos benefícios oferecidos. Tomadas inteligentes baseadas em Wi-Fi geralmente apresentam um custo unitário baixo. Dispositivos Zigbee/Z-Wave podem ter um custo unitário similar ou ligeiramente superior, mas o custo adicional do hub deve ser considerado. O desenvolvimento de hardware customizado, especialmente para produção em pequena escala ou prototipagem única, tende a ser a opção mais cara em termos de componentes e, principalmente, tempo de desenvolvimento.
- **Disponibilidade de API e Ecossistema de Desenvolvimento:** Para este projeto, a capacidade de acessar programaticamente os dados de consumo e, idealmente, controlar os dispositivos é fundamental. Uma API bem documentada, estável e que ofereça os endpoints necessários é um requisito crítico. Um ecossistema de desenvolvimento ativo, com bibliotecas de cliente, fóruns de suporte e exemplos de código, acelera significativamente o desenvolvimento do software de coleta e análise.

Após ponderar esses critérios e as características das alternativas, a linha de tomadas inteligentes *Tuya Smart Wi-Fi* foi selecionada. A decisão baseou-se em sua combinação favorável de: (a) baixo custo de aquisição e ampla disponibilidade no mercado; (b) facilidade de instalação e configuração (*plug and play*) diretamente na rede Wi-Fi existente – um aspecto particularmente relevante para a residência alvo do estudo, com 62m², onde não há necessidade de replicação de sinal – alinhando-se com o critério de compatibilidade; (c) acesso a uma API de nuvem (*Tuya Cloud API*) que, embora com suas complexidades, permite a extração programática de dados de consumo (potência, corrente, tensão) e o envio de comandos de controle, atendendo ao critério de disponibilidade de API; e (d) modularidade, permitindo a fácil expansão do

sistema para monitorar múltiplos dispositivos. Adicionalmente, como o foco principal deste trabalho reside nas integrações de software e na análise de dados, e não no desenvolvimento de hardware proprietário, a escolha por uma solução comercial com API robusta como a da Tuya tornou-se ainda mais estratégica. Embora a precisão exata e a granularidade dos dados pudessem variar e exigir validação, a plataforma Tuya representou o melhor compromisso entre os critérios avaliados para a rápida prototipagem e desenvolvimento de um sistema funcional conforme os objetivos deste PFC. Estas tomadas oferecem integração com a plataforma *Tuya Cloud* e suas APIs, garantindo flexibilidade e escalabilidade para o sistema proposto.

3.1.1 Seleção de Ferramentas de Software e Plataformas de Desenvolvimento

Além da escolha do hardware de monitoramento, a seleção das ferramentas de software e plataformas de desenvolvimento foi crucial para a arquitetura do sistema de coleta, processamento, análise e visualização de dados, bem como para a orquestração das tarefas automatizadas. As decisões foram pautadas pela adequação aos requisitos do projeto, facilidade de integração, robustez e o ecossistema de suporte disponível.

Linguagem de Programação: Python

A escolha da linguagem de programação principal recaiu sobre o Python. Esta decisão foi motivada por diversos fatores:

- **Vasto Ecossistema de Bibliotecas:** Python possui um repositório extenso de bibliotecas maduras e amplamente utilizadas para ciência de dados (e.g., Pandas, NumPy, SciPy, Scikit-learn), desenvolvimento web (e.g., Flask, Django), e interação com APIs, que são diretamente aplicáveis às necessidades de manipulação de dados energéticos, modelagem estatística e desenvolvimento do backend do sistema.
- **Sintaxe Clara e Produtividade:** A sintaxe de Python é conhecida por sua legibilidade e concisão, o que acelera o ciclo de desenvolvimento e facilita a manutenção do código – aspectos importantes no contexto de um PFC com prazos definidos.
- **Comunidade Ativa e Documentação Abundante:** A grande e ativa comunidade de desenvolvedores Python garante farto material de aprendizado, documentação e soluções para problemas comuns, reduzindo o tempo gasto em depuração e pesquisa.

- **Facilidade de Integração com APIs Externas:** A disponibilidade de bibliotecas como `requests` e conectores específicos, como o `tuya-connector-python` utilizado neste projeto, simplifica enormemente a comunicação com APIs de terceiros, incluindo a Tuya Cloud API, permitindo uma rápida implementação da coleta de dados e envio de comandos.
- **Integração com Outras Ferramentas:** Python integra-se facilmente com diversas outras tecnologias e plataformas, incluindo bancos de dados, serviços de nuvem e as ferramentas de orquestração e frontend selecionadas para este projeto.

Foram consideradas algumas alternativas ao Python para o desenvolvimento principal:

- **Node.js (JavaScript/TypeScript):** Considerado por sua eficiência em operações assíncronas e de I/O, o que é relevante para interações com APIs. Contudo, para as necessidades de análise de dados e modelagem estatística do projeto, o ecossistema de bibliotecas Python é mais extenso e maduro.
- **Java ou C#:** Ponderadas por sua robustez em aplicações de larga escala e forte tipagem. No entanto, para um projeto de PFC com foco em rápida prototipagem e análise de dados, o ciclo de desenvolvimento em Python é geralmente mais ágil, e a complexidade adicional dessas linguagens não traria benefícios proporcionais aos objetivos.

No entanto, Python ofereceu o melhor compromisso entre poder analítico, velocidade de desenvolvimento e disponibilidade de bibliotecas específicas para as tarefas centrais do projeto, superando as alternativas no contexto deste PFC.

Biblioteca Pandas e Estrutura DataFrame

A biblioteca Pandas é uma ferramenta fundamental no ecossistema Python para manipulação e análise de dados. Ela introduz estruturas de dados de alto desempenho e fáceis de usar, sendo a principal delas o DataFrame.

- **DataFrame Pandas:** Um DataFrame é uma estrutura de dados tabular bidimensional, semelhante a uma planilha ou tabela SQL, com colunas rotuladas e linhas indexadas. Ele é capaz de armazenar dados de tipos heterogêneos (numéricos, strings, booleanos, etc.) em suas colunas. Os DataFrames oferecem uma vasta gama de funcionalidades para limpeza, transformação, análise e visualização de dados, incluindo:
 - Leitura e escrita de dados em diversos formatos (CSV, Excel, JSON, SQL, Parquet, entre outros).

- Seleção e filtragem de dados com base em rótulos de colunas, índices de linhas ou condições lógicas.
- Tratamento de dados ausentes (e.g., preenchimento ou remoção).
- Operações de agrupamento (*group by*) para realizar agregações (soma, média, contagem, etc.) sobre subconjuntos de dados.
- Junção e mesclagem de múltiplos DataFrames (semelhante a JOINS em SQL).
- Transformações de dados, como adição ou modificação de colunas.
- Análise de séries temporais.
- Integração com outras bibliotecas de visualização (como Matplotlib e Plotly) e de modelagem estatística (como Scikit-learn).

A flexibilidade e o poder computacional dos DataFrames Pandas tornam-nos uma escolha padrão para cientistas de dados e engenheiros que trabalham com dados tabulares em Python. No contexto deste projeto, Pandas foi extensivamente utilizado para carregar o mapeamento de dispositivos, auxiliar no pré-processamento dos dados coletados pela API Tuya (como visto na pipeline Dagster, Seção 3.2.2), e na preparação dos dados para visualização na interface Streamlit (Seção 3.2.3).

Plataforma de Interface de Usuário: Streamlit

Para a construção da interface de usuário (UI) destinada à visualização dos dados e interação com o sistema de controle, a plataforma Streamlit foi selecionada. Os principais motivos para esta escolha foram:

- **Desenvolvimento Rápido em Python Puro:** Streamlit permite criar aplicações web interativas utilizando apenas Python, eliminando a necessidade de proficiência em tecnologias frontend complexas como HTML, CSS, JavaScript e frameworks associados (e.g., React, Angular, Vue.js). Isso permitiu focar nos aspectos de análise de dados e lógica de controle, tornando-o ideal para pesquisadores ou engenheiros que desejam criar ferramentas de dados sem se aprofundar em frameworks web tradicionais, uma vantagem considerável no contexto de um PFC.
- **Ideal para Aplicações de Dados e Prototipagem Rápida:** A biblioteca é projetada especificamente para transformar scripts de análise de dados em aplicações web interativas com poucas linhas de código. Sua facilidade de implementação de componentes interativos (widgets) como seletores de data, menus suspensos para seleção de dispositivos, e a renderização direta de gráficos a

partir de DataFrames Pandas (utilizando bibliotecas como Plotly e Matplotlib de forma transparente), permitiu a criação de uma interface rica e responsiva com um esforço de codificação frontend mínimo. A capacidade de usar o decorador `@st.cache_data` para otimizar o recarregamento de dados e o objeto `st.session_state` para gerenciar estados e interações de usuário complexas, como formulários multi-etapas para configuração de cenas, foram cruciais para construir uma experiência de usuário robusta. A reatividade intrínseca do Streamlit, onde a interface é atualizada automaticamente em resposta a interações com widgets, simplificou o gerenciamento de estado da UI, embora tenha exigido atenção ao design para otimizar o desempenho em visualizações de dados mais complexas, utilizando as ferramentas de cache e estado de forma estratégica. Essa abordagem permitiu que o foco permanecesse na lógica da aplicação e na análise dos dados, em vez de em complexidades de desenvolvimento web tradicional, resultando em um ciclo de prototipagem e iteração muito mais rápido. Adicionalmente, a simplicidade com que aplicações Streamlit podem ser compartilhadas ou implantadas (e.g., utilizando Streamlit Community Cloud ou containerização com Docker) foi considerada um benefício potencial, embora a implantação em larga escala não fosse um objetivo primário deste PFC.

- **Prototipagem Ágil:** A simplicidade do Streamlit facilita a prototipagem rápida e a iteração sobre o design da interface, o que foi valioso para refinar a experiência do usuário ao longo do desenvolvimento.

Para a construção da interface de usuário, foram consideradas algumas alternativas ao Streamlit:

- **Dash (Plotly):** Outra biblioteca Python popular para dashboards analíticos, o Dash oferece um alto grau de customização e controle sobre a interface. No entanto, geralmente apresenta uma curva de aprendizado e uma complexidade de código um pouco superiores em comparação com o Streamlit, especialmente para a rápida prototipagem desejada neste PFC.
- **Solução Full-Stack Tradicional (e.g., Flask/Django + React/Vue/Angular):** O desenvolvimento de uma interface utilizando um framework backend (como Flask ou Django) em conjunto com um framework frontend moderno (como React, Vue.js ou Angular) proporcionaria máxima flexibilidade e escalabilidade. Contudo, esta abordagem exigiria proficiência em múltiplas tecnologias (HTML, CSS, JavaScript, além do backend Python) e um tempo de desenvolvimento significativamente maior, o que extrapolaria o escopo prático e os recursos disponíveis para a interface de usuário deste projeto.

Streamlit representou, portanto, a solução mais eficiente para entregar uma UI funcional e informativa dentro das restrições do PFC, priorizando a velocidade de desenvolvimento e a facilidade de uso para um projeto centrado em Python.

Adicionalmente, para garantir que a aplicação Streamlit pudesse carregar e processar dados analíticos de forma eficiente, a escolha do formato de arquivo para a camada de dados de staging (`app/data/staging/`) foi uma consideração importante. Optou-se pelo formato Parquet, e esta decisão é justificada pelas seguintes vantagens em relação a outros formatos de arquivo comuns:

- **Armazenamento Colunar:** Parquet é um formato de armazenamento colunar. Isso significa que os dados de cada coluna são armazenados juntos, em vez de linha por linha como em formatos como CSV ou JSON. Para consultas analíticas, que frequentemente acessam apenas um subconjunto de colunas de uma tabela larga, o armazenamento colunar é significativamente mais eficiente, pois permite ler apenas os dados das colunas necessárias, reduzindo drasticamente a quantidade de I/O (entrada/saída) e acelerando as consultas.
- **Compressão Eficiente:** Parquet suporta diversos algoritmos de compressão (e.g., Snappy, Gzip, ZSTD) que podem ser aplicados por coluna, aproveitando a similaridade de dados dentro de uma coluna para alcançar altas taxas de compressão. Isso resulta em menor uso de espaço de armazenamento e menor volume de dados a serem lidos do disco ou transferidos pela rede. Formatos como CSV e JSON, por padrão, não são comprimidos ou oferecem compressão menos granular e eficiente.
- **Desempenho em Consultas Analíticas:** Devido ao seu layout colunar e à capacidade de codificação e compressão otimizadas, Parquet oferece desempenho superior para cargas de trabalho analíticas (OLAP). Operações como filtragem, agregações e projeções de colunas são muito mais rápidas em comparação com formatos orientados a linha. Além disso, Parquet suporta técnicas como *predicate pushdown*, onde os filtros podem ser aplicados diretamente na camada de leitura do arquivo, evitando o carregamento de dados desnecessários na memória.
- **Suporte a Tipos de Dados Complexos e Evolução de Esquema:** Parquet possui um rico sistema de tipos de dados, incluindo estruturas aninhadas, e oferece bom suporte para a evolução do esquema (e.g., adição de novas colunas) ao longo do tempo, o que é crucial em sistemas de dados que mudam.
- **Ampla Integração no Ecossistema de Dados:** O formato Parquet é amplamente adotado e suportado por praticamente todas as principais ferramentas e

frameworks de processamento de dados, incluindo Apache Spark, Pandas, Dask, e sistemas de consulta como DuckDB (utilizado neste projeto), Presto e Athena. Essa interoperabilidade facilita a integração em diversas etapas da pipeline de dados.

Em comparação com alternativas comuns:

- **CSV (Comma-Separated Values):** Embora simples e legível por humanos, o CSV é ineficiente para grandes volumes de dados analíticos. Ele é orientado a linha, não possui informações de tipo de dados embutidas (requerendo inferência), e geralmente não é comprimido por padrão, levando a arquivos maiores e leituras mais lentas.
- **JSON (JavaScript Object Notation):** Ideal para dados semiestruturados ou aninhados e também legível por humanos. No entanto, é um formato verboso, orientado a linha, e o parsing de arquivos JSON grandes pode ser computacionalmente intensivo. Para consultas analíticas sobre dados tabulares, é consideravelmente menos performático que Parquet.
- **Apache Avro:** Outro formato popular, especialmente em pipelines de streaming de dados (e.g., com Kafka). Avro é um formato binário orientado a linha que armazena o esquema junto com os dados, oferecendo serialização compacta e boa evolução de esquema. Embora seja uma melhoria em relação a CSV/JSON para muitos casos de uso, para cargas de trabalho puramente analíticas e armazenamento em data lakes, o formato colunar do Parquet geralmente proporciona melhor desempenho de consulta.

Portanto, a escolha pelo Parquet para os dados na camada de staging foi estratégica para otimizar o desempenho das análises e a eficiência do armazenamento, beneficiando indiretamente a responsividade da aplicação Streamlit ao consultar esses dados.

Plataforma de Orquestração de Dados: Dagster

Para a orquestração dos fluxos de dados (data pipelines) – incluindo a coleta periódica de dados da API Tuya, o processamento e armazenamento desses dados, e o agendamento da execução de cenários de automação – a ferramenta Dagster foi escolhida. A decisão baseou-se nos seguintes pontos:

- **Abordagem Centrada em Ativos de Dados (Data Assets):** Dagster adota uma filosofia moderna de engenharia de dados, tratando os pipelines como grafos

de ativos de dados (*Software-defined Assets*). Esta abordagem permitiu modelar explicitamente as dependências entre os dados brutos extraídos da API Tuya (um ativo) e os dados processados e armazenados em formato Parquet na camada de staging (outro ativo dependente). A definição destes ativos diretamente em Python, onde cada ativo pode representar uma tabela, um conjunto de arquivos ou um modelo de machine learning, e a capacidade de visualizar o grafo de dependências (DAG) e o histórico de materializações no Dagit, foram particularmente úteis para gerenciar a evolução da pipeline de dados, aumentar a observabilidade e facilitar a depuração.

- **Desenvolvimento em Python Nativo:** Sendo uma ferramenta Python-nativa, Dagster integrou-se perfeitamente com o restante do código do projeto, permitindo que toda a lógica da pipeline fosse escrita na mesma linguagem, aproveitando as bibliotecas e o conhecimento existentes.
- **Interface de Usuário (Dagit) e Observabilidade:** A interface web Dagit, fornecida pelo Dagster, oferece excelente observabilidade sobre as execuções dos pipelines, o status de materialização dos ativos, logs detalhados por execução e o gerenciamento de agendamentos. Para um projeto de PFC, essa capacidade de visualização e depuração interativa acelera significativamente o desenvolvimento, permitindo identificar e corrigir problemas rapidamente. A visualização do grafo de ativos e suas dependências também auxilia na compreensão da arquitetura da pipeline.
- **Gerenciamento de Configuração com Recursos (Resources):** A capacidade do Dagster de definir e gerenciar **Resources** (como conexões a bancos de dados ou credenciais de API) de forma desacoplada da lógica dos ativos ou ops foi um diferencial. No projeto, isso foi utilizado para configurar o acesso à API Tuya (via **TuyaCredentials**) e a conexão com DuckDB (via **DuckDBResource**), promovendo um código mais limpo, testável e facilitando o gerenciamento de configurações entre ambientes de desenvolvimento e uma potencial futura produção.
- **Agendamento e Gerenciamento de Dependências Robustos:** Dagster possui um sistema robusto para definir agendamentos (*schedules*) e gerenciar as dependências entre as diferentes etapas (*ops* e *assets*) da pipeline, garantindo que as tarefas sejam executadas na ordem correta e em intervalos programados.
- **Testabilidade e Confiabilidade:** A arquitetura do Dagster, com sua separação entre a lógica de negócios (definida em *ops* e *assets*) e o motor de execução, facilita a escrita de testes unitários para os componentes da pipeline de forma isolada.

Adicionalmente, o Dagit permite reexecuções parciais de pipelines a partir de etapas específicas que falharam, o que é crucial para a depuração eficiente e para garantir a resiliência do processamento de dados em caso de problemas transitórios.

- **Facilidade de Desenvolvimento e Depuração Local:** A experiência de desenvolvimento local com Dagster, utilizando o comando `dagster dev`, é direta e eficiente. Este comando inicia tanto a interface Dagit quanto o daemon de execução, permitindo que o desenvolvedor execute, visualize e depure pipelines integralmente em seu ambiente local antes de qualquer implantação, agilizando o ciclo de desenvolvimento iterativo característico de um PFC.

Para a orquestração dos fluxos de dados, foram avaliadas algumas alternativas ao Dagster:

- **Apache Airflow:** Uma ferramenta de orquestração de fluxos de trabalho muito popular e poderosa, com uma grande comunidade e um vasto conjunto de funcionalidades. No entanto, Airflow pode apresentar uma curva de aprendizado mais íngreme e uma complexidade de configuração e gerenciamento de infraestrutura (e.g., scheduler, webserver, workers, metadatabase) maior, o que poderia ser excessivo para o escopo e a simplicidade desejada para um projeto individual de PFC.
- **Prefect:** Outra plataforma moderna de orquestração de fluxos de dados, que compartilha muitas das características e da filosofia de desenvolvimento em Python do Dagster. A escolha entre Dagster e Prefect pode, por vezes, depender de preferências específicas ou de familiaridade prévia, mas ambas representam avanços significativos em relação a ferramentas mais antigas.
- **Scripts Python Agendados via Cron:** Uma abordagem mais simples seria utilizar scripts Python para executar as tarefas de coleta e processamento, agendando-os através de um utilitário do sistema como o cron (em sistemas Unix-like) ou o Agendador de Tarefas (no Windows). Embora simples de implementar para tarefas básicas, esta abordagem carece de funcionalidades importantes para pipelines de dados mais complexas, como gerenciamento de dependências entre tarefas, monitoramento centralizado, logging robusto, interface de usuário para observabilidade, capacidade de reexecução granular de tarefas falhas e versionamento de pipelines.

Dagster foi percebido como o melhor equilíbrio entre modernidade (com sua abordagem centrada em ativos de dados), facilidade de uso para um desenvolvedor Python,

robustez para gerenciar dependências e agendamentos, e excelentes funcionalidades de observabilidade através da interface Dagit, adequando-se bem às necessidades específicas deste trabalho sem a sobrecarga de ferramentas mais complexas.

Banco de Dados Analítico Embutido: DuckDB

A seleção de uma ferramenta adequada para o armazenamento intermediário e processamento analítico dos dados coletados pelos sensores Tuya foi uma etapa crucial. Para o processamento e transformação destes dados, especialmente na transição da camada bruta (raw) para a camada de preparação (staging) e para consultas analíticas subsequentes, a escolha recaiu sobre o DuckDB. Trata-se de um sistema de gerenciamento de banco de dados analítico (OLAP) embutido, projetado para ser rápido, confiável e fácil de usar.

Os principais motivos para a seleção do DuckDB para lidar com os dados provenientes dos sensores foram:

- **Alto Desempenho para Cargas de Trabalho Analíticas:** DuckDB é otimizado para consultas analíticas complexas e agregações, utilizando processamento vetorializado e colunar. Isso é particularmente vantajoso ao lidar com séries temporais de dados de consumo energético, onde agregações e junções são operações comuns.
- **Banco de Dados Embutido (In-Process):** Diferentemente de bancos de dados tradicionais baseados em servidor, o DuckDB roda diretamente no mesmo processo da aplicação Python que consome os dados dos sensores. Isso elimina a necessidade de configurar, gerenciar e manter um servidor de banco de dados separado, simplificando significativamente a arquitetura de deployment e reduzindo a sobrecarga operacional, um fator importante no contexto de um PFC.
- **Integração Profunda com Python e Pandas:** DuckDB oferece uma API Python intuitiva que permite a execução de consultas SQL diretamente sobre DataFrames Pandas (que podem conter os dados dos sensores) e a fácil conversão de resultados de consultas SQL de volta para DataFrames. Essa interoperabilidade fluida foi crucial, dado que Pandas já era uma ferramenta central para manipulação de dados no projeto.
- **Capacidade de Consultar Diretamente Diversos Formatos de Arquivo:** Uma característica distintiva do DuckDB é sua habilidade de executar consultas SQL diretamente sobre arquivos em formatos como Parquet, CSV e JSON — formatos nos quais os dados dos sensores podem ser inicialmente armazenados

ou processados — sem a necessidade de importação prévia para tabelas internas. Isso foi explorado na pipeline de dados para ler os arquivos JSON da camada bruta e para interagir com os arquivos Parquet da camada de preparação.

- **Sintaxe SQL Completa e Moderna:** DuckDB suporta uma vasta gama do padrão SQL, incluindo funções de janela, expressões de tabela comuns (CTEs) e tipos de dados complexos, facilitando a escrita de transformações de dados sofisticadas sobre os dados dos sensores.
- **Leveza e Facilidade de Instalação:** A instalação do DuckDB é simples (e.g., via pip) e não requer dependências complexas, tornando-o fácil de integrar em qualquer ambiente Python para processamento dos dados coletados.

Para o armazenamento e processamento dos dados dos sensores, foram consideradas algumas alternativas ao DuckDB:

- **SQLite:** Também é um banco de dados embutido, amplamente utilizado e simples. No entanto, SQLite é primariamente um banco de dados transacional (OLTP) e, embora capaz de lidar com algumas consultas analíticas, não possui o mesmo nível de otimização para cargas de trabalho OLAP complexas e grandes volumes de dados de sensores que o DuckDB oferece.
- **Manipulação de Dados Exclusivamente com Pandas:** Pandas é uma biblioteca poderosa para análise de dados em Python. Contudo, para transformações de dados muito complexas ou operações que envolvem grandes datasets de sensores que podem não caber inteiramente na memória, a execução de consultas SQL via DuckDB pode ser mais performática, mais eficiente em termos de uso de memória (devido ao seu motor de execução otimizado e capacidade de realizar operações out-of-core) e, para alguns desenvolvedores, mais expressiva para certas lógicas de transformação.
- **Bancos de Dados Baseados em Servidor (e.g., PostgreSQL, MySQL):** Soluções como PostgreSQL ou MySQL são sistemas de gerenciamento de banco de dados relacionais (RDBMS) robustos e completos. No entanto, eles introduzem a necessidade de um serviço de servidor separado, aumentando a complexidade da configuração e do gerenciamento da infraestrutura para os dados dos sensores. Para o escopo de um PFC focado na prototipagem de um sistema de análise de dados, o overhead de um RDBMS completo foi considerado desnecessário.

Dessa forma, DuckDB foi selecionado para o tratamento dos dados dos sensores por oferecer o melhor equilíbrio entre alto desempenho analítico, facilidade de integração

com o ecossistema Python/Pandas, simplicidade de configuração e a capacidade de lidar eficientemente com os formatos de arquivo e as necessidades de processamento de dados do projeto, sem a complexidade de um banco de dados baseado em servidor. Sua utilização como recurso no Dagster ([DuckDBResource](#)), conforme detalhado na Seção 3.2.2 ao descrever a implementação da pipeline de dados, permitiu uma integração limpa para as transformações na camada de staging.

3.2 Desenvolvimento do Protótipo

Com a seleção das tomadas inteligentes *Tuya Smart Wi-Fi* como hardware de medição e as plataformas Python, Dagster e Streamlit como pilares do software, iniciou-se o desenvolvimento do protótipo funcional. Este sistema foi concebido para integrar hardware e software de forma coesa, permitindo o monitoramento detalhado do consumo energético, a aplicação de técnicas de Controle Estatístico de Processo (CEP) e a interação do usuário por meio de uma interface intuitiva. O desenvolvimento foi estruturado em etapas iterativas, abrangendo desde a configuração inicial dos sensores até a implementação das lógicas de controle e análise de dados, conforme detalhado nas subseções seguintes. O objetivo central desta fase foi materializar a arquitetura planejada em um sistema operacional, capaz de coletar, processar, analisar e apresentar dados energéticos de forma eficaz.

3.2.1 Seleção e Configuração de Sensores

A etapa inicial do desenvolvimento prático do protótipo consistiu na aquisição e configuração de um conjunto inicial de cinco tomadas inteligentes da plataforma *Tuya Smart Wi-Fi*. A seleção dos modelos específicos dentro do vasto ecossistema Tuya considerou: a confirmação explícita do suporte à medição de energia (potência, corrente, tensão) nas especificações do produto ou em documentação comunitária; uma capacidade de corrente nominal compatível com eletrodomésticos comuns (tipicamente 10A ou 16A); e um formato físico que não obstruísse tomadas adjacentes. A compatibilidade com a API Tuya para extração de dados foi verificada preliminarmente através da documentação da *Tuya Developer Platform* e da análise de bibliotecas Python de código aberto que interagem com o ecossistema Tuya. Para este projeto, foram utilizados modelos genéricos rotulados como "Smart Plug Wi-Fi", frequentemente equipados com chipsets de medição como o BL0937 ou similares, capazes de reportar potência instantânea (tipicamente com granularidade de 0.1W), corrente (em miliampères) e tensão (com granularidade de 0.1V). Estes dispositivos também contam com um botão físico para

controle manual (liga/desliga), servindo como um backup ao controle via software. É importante notar que, embora a precisão e a granularidade desses sensores de nível de consumidor sejam adequadas para os objetivos deste PFC – identificar padrões de consumo e aplicar CEP em um contexto residencial – elas não se comparam à precisão de medidores de energia de grau de faturamento ou equipamentos de laboratório. A escolha priorizou o custo-benefício, a facilidade de integração via API e a capacidade de prototipagem rápida, alinhando-se com as restrições e objetivos de um trabalho de conclusão de curso.

O processo de configuração de cada tomada seguiu o procedimento padrão estabelecido pelo fabricante:

1. **Instalação Física:** Cada tomada foi conectada a uma fonte de alimentação elétrica padrão (127V ou 220V, conforme a especificação do modelo e da rede local), observando-se a capacidade máxima de carga do dispositivo.
2. **Modo de Emparelhamento:** As tomadas foram colocadas em modo de emparelhamento, geralmente pressionando um botão físico no dispositivo por alguns segundos até que um LED indicador (frequentemente azul ou branco) começasse a piscar rapidamente, sinalizando que o dispositivo estava pronto para ser descoberto.
3. **Conexão via Aplicativo Móvel:** Utilizando um aplicativo móvel fornecido pela Tuya (como o "Tuya Smart" ou "Smart Life", disponíveis para Android e iOS), iniciou-se o processo de adição de um novo dispositivo. Um cuidado importante nesta etapa foi garantir que tanto o smartphone executando o aplicativo quanto a tomada inteligente estivessem operando na banda de 2.4GHz da rede Wi-Fi, pois muitos dispositivos IoT, incluindo estas tomadas, não são compatíveis com a banda de 5GHz. O aplicativo detecta a tomada em modo de emparelhamento e guia o usuário na conexão desta à rede Wi-Fi local (requerendo SSID e senha). Após o pareamento bem-sucedido, foi verificado no próprio aplicativo se havia atualizações de firmware disponíveis para as tomadas, aplicando-as quando necessário para assegurar a estabilidade e o acesso às funcionalidades mais recentes da API.
4. **Associação à Conta Tuya Cloud e Verificação Inicial:** Durante o processo de configuração via aplicativo, cada tomada é automaticamente associada à conta do usuário na plataforma *Tuya Cloud*. Esta associação é fundamental, pois é através desta conta que as APIs da Tuya autenticam e autorizam o acesso aos dados dos dispositivos. Cada dispositivo recebe um identificador único (Device ID) na

plataforma. Ainda no aplicativo móvel, foi possível atribuir nomes descritivos a cada tomada (e.g., "Geladeira", "TV Sala") e verificar seu status de conexão e o reporte inicial de dados de energia, confirmando o funcionamento básico antes de prosseguir com a integração via API.

Para viabilizar o acesso programático a estes dados via API, foi necessário um passo adicional de configuração na *Tuya Developer Platform*. Este processo envolveu a criação de uma conta de desenvolvedor, o registro de um novo projeto do tipo "Cloud Development", e a vinculação da conta do aplicativo móvel (e.g., "Tuya Smart" ou "Smart Life"), onde os dispositivos foram inicialmente pareados, a este projeto na plataforma de desenvolvimento. Somente após esta vinculação e a autorização dos serviços de API necessários (como "IoT Core" e "Basic Device Management"), foram obtidas as credenciais de API (`ACCESS_ID` e `ACCESS_SECRET`) indispensáveis para a autenticação das chamadas de software à nuvem da Tuya.

Uma vez configuradas e conectadas à rede Wi-Fi, as tomadas inteligentes iniciaram o monitoramento do consumo energético dos aparelhos a elas conectados. Internamente, estes dispositivos contêm chipsets de medição de energia (e.g., modelos da série BL09xx da Shanghai Belling ou similares) que amostram continuamente a tensão e a corrente. Estes chipsets calculam a potência e outros parâmetros, que são então reportados à plataforma *Tuya Cloud* através de *data points* (DPs) específicos. Para dispositivos de medição de energia, DPs comuns incluem códigos como `cur_power` para potência instantânea (geralmente em décimos de Watt, W/10), `cur_current` para corrente (em miliampères, mA), e `cur_voltage` para tensão (em décimos de Volt, V/10), além de DPs para o estado do relé (e.g., `switch_1`). A interpretação correta destes DPs e seus respectivos fatores de escala é crucial para a obtenção de leituras precisas. Para o modelo de tomada inteligente *Tuya Smart Wi-Fi* selecionado para este projeto, a captura de dados de consumo energético (potência, corrente, tensão) é realizada e enviada para a nuvem da Tuya em intervalos de uma hora. Essas informações são armazenadas na plataforma Tuya e ficam disponíveis para consulta via API por um período de 7 dias. Após esse período, os dados mais antigos são sobrescritos ou descartados pela plataforma. Esta característica de retenção de dados da API Tuya impõe a necessidade de um sistema de coleta e armazenamento persistente, como a pipeline de dados implementada com Dagster (detalhada na Seção 3.2.2), para garantir que um histórico completo do consumo energético seja mantido para análises de longo prazo e para a construção da linha de base do CEP. Embora a granularidade horária possa não capturar flutuações de consumo de curtíssimo prazo, ela foi considerada suficiente para os objetivos deste projeto, que focam na identificação de padrões de consumo e anomalias em uma escala de tempo mais ampla, adequada para o controle estatístico

de processo em nível residencial. A frequência exata de envio e a granularidade dos dados podem variar entre diferentes modelos de tomadas e versões de firmware, mas a especificação de coleta horária e retenção de 7 dias foi confirmada para os dispositivos utilizados. Estes dados de consumo, acessíveis através da *Tuya Developer Platform* e associados a um Device ID específico, formam a base para a coleta e análise programática. O mapeamento entre o Device ID (fornecido pela plataforma Tuya) e um nome descritivo para o aparelho/local monitorado foi registrado manualmente em um arquivo de configuração JSON, localizado em `app/data/device_mapping.json`. Este arquivo é essencial para o enriquecimento semântico dos dados coletados, permitindo que a aplicação apresente informações de forma mais compreensível para o usuário. A estrutura do arquivo segue um padrão de dicionário onde a chave é o Device ID e o valor é um objeto contendo, no mínimo, o nome do dispositivo. Por exemplo:

```
{
  "bfxxxxxxxxxxxxxxxxSMARTPLUGID1": {
    "name": "Geladeira Cozinha",
    "default_code": "switch_1"
  },
  "bfyyyyyyyyyyyyyySMARTPLUGID2": {
    "name": "Microondas Sala",
    "default_code": "switch_1"
  }
}
```

O campo `default_code` refere-se ao código do *data point* (DP) padrão utilizado para controlar o estado de ligar/desligar do dispositivo, que tipicamente é `"switch_1"` para tomadas simples.

3.2.2 Arquitetura e Desenvolvimento do Software de Coleta, Processamento e Controle

O componente de software do sistema é fundamental para transformar os dados brutos de energia em informações acionáveis e para permitir o controle automatizado dos dispositivos. Seu desenvolvimento foi centrado na criação de um backend robusto para a coleta e processamento de dados, na implementação de lógicas de controle para automação de cenas, e na orquestração eficiente desses processos. A comunicação com as tomadas inteligentes *Tuya Smart Wi-Fi* foi realizada exclusivamente através da *Tuya Cloud API*, utilizando o conector Python `tuya-connector-python`. Esta biblioteca encapsula as complexidades da autenticação (baseada em HMAC-SHA256

e tokens de acesso) e das chamadas HTTP aos diversos endpoints da API, como os de consulta de status de dispositivos (`GET /v1.0/devices/{device_id}/status`) e envio de comandos (`POST /v1.0/devices/{device_id}/commands`). A arquitetura de software foi projetada para ser modular, escalável e de fácil manutenção, utilizando Python como linguagem principal e a plataforma Dagster para a orquestração dos fluxos de dados e tarefas agendadas.

Implementação da Pipeline de Dados com Dagster

A gestão do fluxo de dados de consumo energético, desde a extração da API Tuya até o armazenamento em um formato otimizado para análise, foi implementada utilizando a plataforma de orquestração Dagster. A escolha do Dagster, como justificado anteriormente, deveu-se à sua abordagem moderna de engenharia de dados, que permite definir pipelines como código Python, monitorar execuções através da interface Dagit, gerenciar dependências complexas entre etapas de processamento (ativos) e programar execuções recorrentes de forma robusta.

A metodologia de implementação da pipeline de dados com Dagster compreendeu os seguintes passos técnicos:

- **Configuração de Recursos (Resources):**
 - **TuyaCredentials:** Um objeto de configuração (`Config`) foi definido para gerenciar as credenciais da API Tuya (`ACCESS_ID`, `ACCESS_SECRET`, `API_ENDPOINT`) e o caminho para o arquivo de mapeamento de dispositivos. Estes valores são carregados a partir de variáveis de ambiente (utilizando `python-dotenv` para desenvolvimento local e `EnvVar` do Dagster para definição formal), garantindo que informações sensíveis não sejam codificadas diretamente no código fonte.
 - **DuckDBResource:** Foi configurado um recurso Dagster para interagir com o DuckDB. Inicialmente, este foi configurado para operar com um banco de dados em memória, adequado para o processamento rápido e transformações dentro do ativo de staging. O DuckDB foi escolhido por sua alta performance em cargas de trabalho analíticas e sua capacidade de consultar diretamente arquivos Parquet e JSON.
- **Definição de Ativos (Assets):** Dois ativos principais foram desenvolvidos para compor a pipeline:
 - **raw_tuya_logs:** Este ativo é responsável pela extração dos dados brutos da API Tuya. Utilizando o `tuya-connector-python` e as credenciais fornecidas

pelo recurso `TuyaCredentials`, o ativo itera sobre a lista de dispositivos mapeados e requisita os logs de status de energia (reportados pelos dispositivos) para cada um. Os dados retornados pela API, geralmente em formato JSON contendo múltiplos registros de *data points* (DPs) como potência, corrente e tensão com seus respectivos timestamps, são então salvos como arquivos JSON individuais. Para organização e otimização de futuras leituras, estes arquivos são armazenados em uma estrutura de diretórios particionada por `device_id` e data (YYYY-MM-DD) no caminho `app/data/raw/`. O ativo retorna o caminho absoluto para o diretório base onde os dados brutos foram salvos.

- `staging_tuya_logs`: Este ativo é dependente do `raw_tuya_logs`, recebendo como entrada o caminho para os dados brutos. Sua principal função é transformar os dados JSON brutos em um formato tabular otimizado (Parquet) e armazená-los na camada de staging. O processo envolve:
 1. Ler todos os arquivos JSON do diretório de dados brutos.
 2. Utilizar o `DuckDBResource` para carregar os dados JSON em tabelas temporárias.
 3. Realizar transformações nos dados, que incluem: conversão do timestamp do evento (originalmente em milissegundos) para um formato de data e hora padrão e mais legível; extração do nome do arquivo de origem para rastreabilidade; e junção com os dados de mapeamento de dispositivos (carregados a partir do arquivo `app/data/device_mapping.json` utilizando Pandas e registrados como uma view no DuckDB) para enriquecer os registros com nomes e locais dos dispositivos.
 4. Derivar uma coluna `event_date` a partir do timestamp do evento, para ser utilizada como chave de particionamento na camada de staging.
 5. Salvar os dados transformados em formato Parquet no diretório `app/data/staging/`, utilizando particionamento por `event_date` (Hive-style partitioning, e.g., `event_date=YYYY-MM-DD/data.parquet`). O formato Parquet foi escolhido por sua eficiência de armazenamento colunar e performance em consultas analíticas.

O ativo retorna o caminho absoluto para o diretório base da camada de staging.

- **Criação de Job (`tuya_processing_job`):** Um job foi definido no Dagster para encapsular os ativos `raw_tuya_logs` e `staging_tuya_logs` e suas dependências

explícitas. O job define a ordem de execução e permite que a pipeline seja executada como uma unidade coesa.

- **Definição de Agendamento (Schedule - hourly_schedule):** Um agendamento foi criado utilizando a funcionalidade de `ScheduleDefinition` do Dagster, configurado com uma expressão cron ("0 * * * *") para executar o `tuya_processing_job` automaticamente a cada hora. Isso garante que os dados de consumo energético sejam coletados e processados regularmente, mantendo o sistema atualizado.
- **Configuração do Ambiente Dagster:** Foram criados os arquivos de configuração padrão do Dagster: `dagster.yaml` (para configurações da instância, como armazenamento de execuções e intermediários) e `workspace.yaml` (para informar ao Dagster onde encontrar as definições de código da pipeline, apontando para o módulo Python que contém as definições de ativos, jobs e schedules, neste caso, `app/assets.py`).

Esta arquitetura de pipeline de dados com Dagster proporciona uma solução robusta, monitorável e automatizada para a ingestão e preparação dos dados de consumo energético, fundamentais para as análises subsequentes e para a interface de usuário.

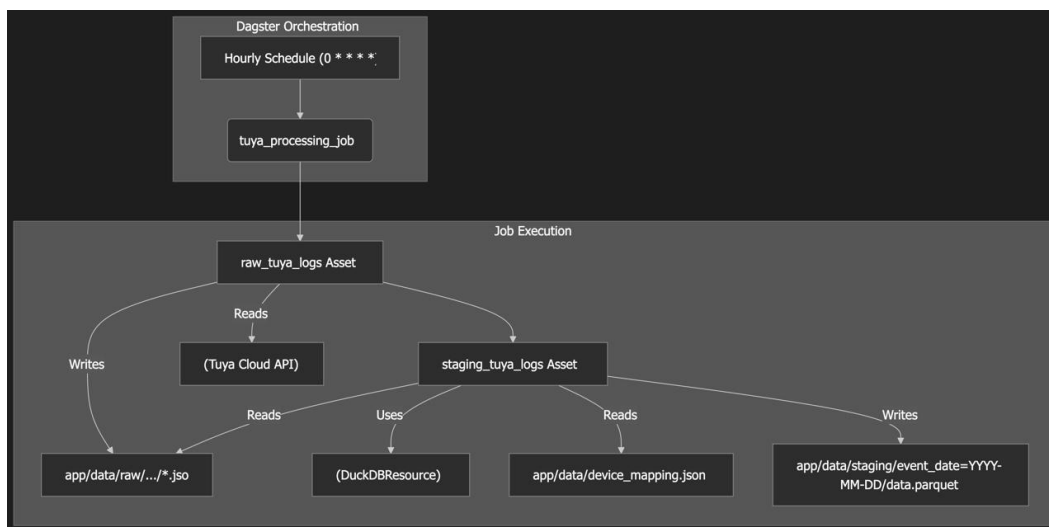


Figura 3.1: Fluxograma da pipeline de processamento de dados de consumo energético com Dagster.

Implementação do Agendamento de Cenas com Dagster

Antes de detalhar a implementação do agendamento de cenas, é relevante contextualizar alguns dos mecanismos de comunicação e segurança empregados na interação com serviços externos, como a API Tuya, mencionada anteriormente. A comunicação com

tais APIs frequentemente envolve conceitos como HMAC-SHA256 para segurança, o paradigma de APIs REST e o uso de chamadas HTTP.

- **HMAC-SHA256:** O HMAC-SHA256 (Hash-based Message Authentication Code using SHA-256) é um mecanismo de autenticação de mensagens que utiliza uma função de hash criptográfica (SHA-256) em conjunto com uma chave secreta. Ele garante tanto a integridade dos dados (que a mensagem não foi alterada em trânsito) quanto a autenticidade (que a mensagem provém de uma fonte que possui a chave secreta), sendo fundamental para a segurança na comunicação com APIs.
- **APIs REST:** As APIs REST (Representational State Transfer) são um estilo arquitetural para projetar aplicações em rede. Elas utilizam requisições HTTP para acessar e manipular representações de recursos web. Cada recurso é identificado por uma URI (Uniform Resource Identifier) única, e as interações são tipicamente realizadas através dos métodos HTTP padrão.
- **Chamadas HTTP:** As chamadas HTTP (Hypertext Transfer Protocol) são o fundamento da comunicação de dados na World Wide Web. Elas consistem em requisições enviadas por um cliente (como o software de coleta de dados) a um servidor (como a Tuya Cloud API) e as respostas correspondentes. Métodos comuns incluem GET (para obter dados), POST (para enviar dados para processamento), PUT (para atualizar um recurso) e DELETE (para remover um recurso).

A automação da execução de cenas de controle de dispositivos (e.g., ligar ou desligar um conjunto de aparelhos em horários pré-determinados ou sob condições específicas) foi implementada utilizando as capacidades de agendamento e execução de lógica Python da plataforma Dagster. Esta abordagem foi escolhida por centralizar a lógica de automação, aproveitar a infraestrutura de monitoramento e logging do Dagster, e permitir uma integração coesa com o restante do sistema de gerenciamento de dados.

A metodologia de implementação envolveu os seguintes componentes técnicos e lógicos:

- **Recurso de API Tuya para Cenas (`tuya_api_resource`):** De forma análoga ao recurso de credenciais utilizado na pipeline de ingestão de dados, um recurso Dagster específico, denominado `tuya_api_resource`, foi configurado. Este recurso é responsável por encapsular de forma segura as credenciais de acesso à API Tuya (`ACCESS_ID`, `ACCESS_SECRET`, `API_ENDPOINT`) e a lógica de instanciação do cliente `tuya-connector-python`. Ao definir este como um recurso, as

credenciais são gerenciadas externamente ao código da operação (idealmente através de variáveis de ambiente ou segredos gerenciados pelo Dagster), e o cliente da API é injetado como uma dependência na operação que executa as cenas. Isso promove boas práticas de segurança e facilita a testabilidade e configuração do sistema.

- **Definição da Operação (Op) `check_and_trigger_scenes_op`:** Esta operação (ou "op" no jargão do Dagster) constitui o núcleo da lógica de execução de cenas. É uma função Python que recebe o `tuya_api_resource` como entrada e executa as seguintes etapas:

1. **Leitura e Validação das Definições de Cena:** O op inicia lendo as definições de cenas, que são persistidas em um arquivo JSON (`app/data/scheduled_scenes.json`). Este arquivo é gerenciado pela interface Streamlit, onde os usuários podem criar, modificar ou excluir cenas. Cada entrada no JSON representa uma cena e contém atributos como: um identificador único da cena, um nome descritivo, uma lista de dispositivos alvo (cada um com seu `Device ID` Tuya), as ações desejadas para cada dispositivo (e.g., ligar/desligar, geralmente representadas por um código de comando como `switch_1` e um valor booleano `true/false`), o horário de execução programado (no formato HH:MM), os dias da semana para ativação (e.g., uma lista de dias como `["seg", "qua", "sex"]`), e um status de recorrência (indicando se a cena é diária, semanal, apenas para os dias especificados, ou de execução única). O op realiza uma validação básica da estrutura de cada cena lida.
2. **Verificação de Agendamento e Condições:** Para cada cena ativa e validada, o op compara o horário e os dias de ativação programados com o momento atual (data e hora corrente). Esta verificação é sensível ao fuso horário configurado no ambiente de execução do Dagster para garantir a precisão. A lógica de comparação verifica se a hora e minuto atuais coincidem com o programado e se o dia da semana atual está na lista de dias de ativação da cena. Para cenas mais complexas, poderiam ser adicionadas verificações de condições (e.g., baseadas em dados de outros sensores ou estados do sistema), embora o escopo inicial tenha focado em agendamentos baseados em tempo.
3. **Lógica de Recorrência e Gerenciamento de Estado para Execução Única:** Uma lógica específica é implementada para tratar cenas não recorrentes. Para estas, após a primeira execução bem-sucedida, o op atualiza o status da cena no arquivo JSON (ou, idealmente, em um mecanismo

de estado mais robusto e transacional, como um pequeno banco de dados ou o próprio sistema de metadados do Dagster, se aplicável) para "executada" ou desativa a cena. Isso previne reativações acidentais em execuções subsequentes do `op`. Para cenas recorrentes (diárias ou semanais nos dias especificados), a lógica simplesmente permite a reavaliação e potencial reativação conforme o agendamento, sem alteração de estado persistente após cada execução.

4. **Envio de Comandos via API Tuya:** Se uma cena é identificada como devida para execução (i.e., todas as condições de horário e dia são satisfeitas e, para cenas não recorrentes, ainda não foi executada), o `op` utiliza o `tuya_api_resource` (o cliente da API Tuya instanciado) para iterar sobre a lista de dispositivos da cena. Para cada dispositivo, ele constrói o payload de comando apropriado (e.g., `{"commands": [{"code": "switch_1", "value": true}]}` para ligar um dispositivo, ou `{"code": "switch_1", "value": false}` para desligar) e envia a requisição para o endpoint de comandos da API Tuya (`POST /v1.0/devices/{device_id}/commands`). O tratamento de erros de comunicação com a API é crucial; são implementadas lógicas de retentativa para falhas transitórias (e.g., timeouts, erros de rede) e logging detalhado para erros persistentes ou respostas de erro da API (e.g., dispositivo offline, comando inválido).
 5. **Logging Detalhado da Execução:** A operação registra informações detalhadas sobre suas ações utilizando o sistema de logging integrado do Dagster (`context.log`). São logados eventos como: o início e fim da verificação de cenas, quais cenas foram avaliadas, quais foram identificadas para acionamento, os Device IDs e comandos enviados para cada dispositivo, e o resultado (sucesso ou falha, incluindo códigos de erro da API) de cada chamada à API Tuya. Este logging é fundamental para a depuração, monitoramento do comportamento do agendador de cenas e para auditoria futura das automações executadas.
- **Criação de Job (`scene_scheduler_job`):** Um job Dagster específico, denominado `scene_scheduler_job`, foi criado para encapsular a operação `check_and_trigger_scenes`. Este job define a unidade de execução que será efetivamente agendada e monitorada pelo Dagster, contendo toda a lógica de verificação e acionamento de cenas.
 - **Definição de Agendamento (Schedule - `scene_execution_schedule`):** Para garantir a verificação frequente e a execução pontual das cenas programadas, um agendamento Dagster (`scene_execution_schedule`) foi configurado. Este uti-

liza uma expressão cron para definir a frequência de execução do `scene_scheduler_job`. Inicialmente, foi definido um intervalo de um minuto ("* * * * *" em notação cron). Esta frequência representa um compromisso entre a responsividade desejada para o sistema de cenas (i.e., cenas serem acionadas próximo ao horário programado) e a carga gerada sobre a API Tuya e o sistema Dagster (evitando chamadas excessivas). A escolha do intervalo pode ser ajustada conforme a necessidade e as limitações da API.

A integração desta funcionalidade de agendamento de cenas no Dagster permite que ela seja tão robusta, monitorável e configurável quanto a pipeline de ingestão de dados, utilizando uma plataforma unificada para todas as tarefas de backend automatizadas e orquestradas do projeto. Isso simplifica a manutenção e a observabilidade do sistema como um todo.

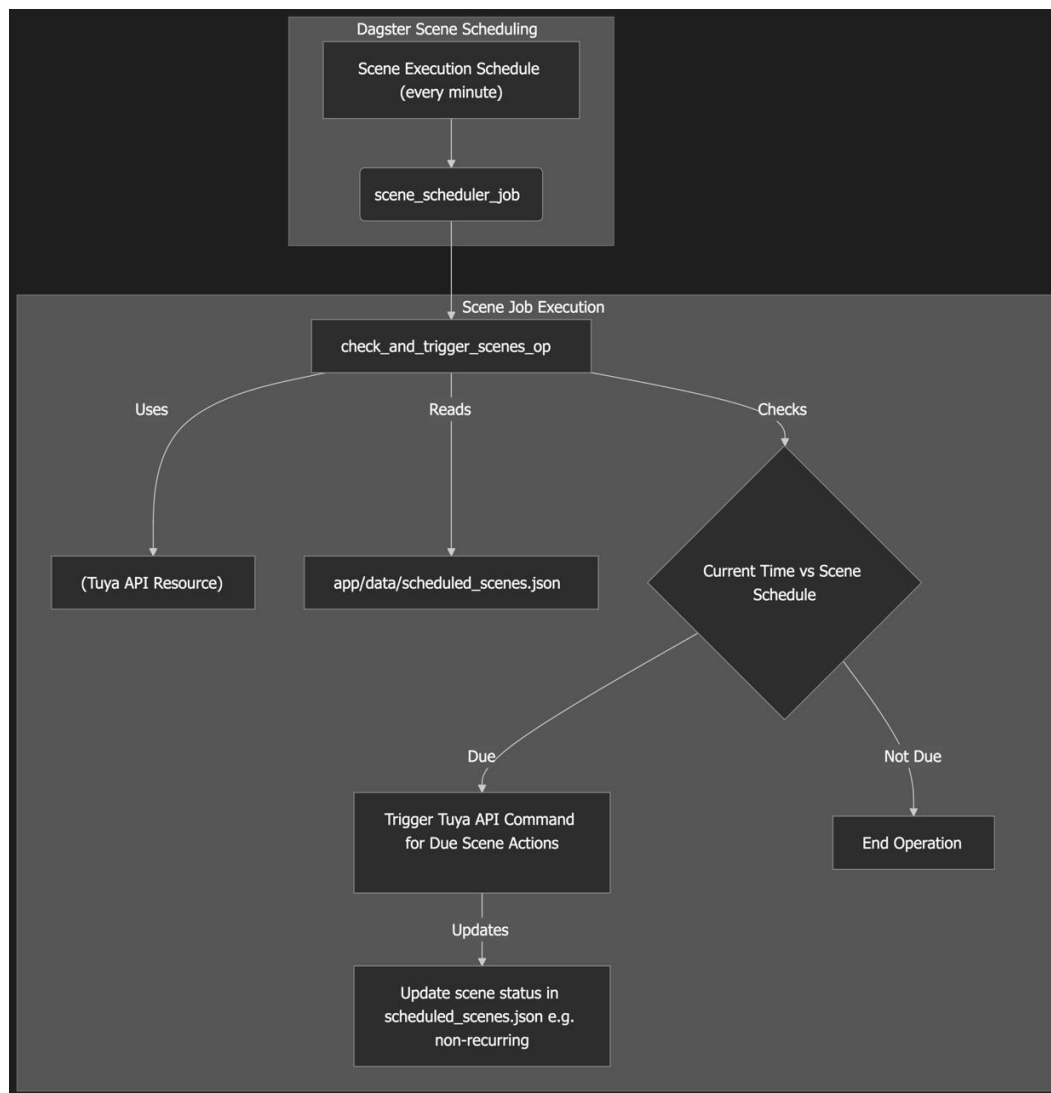


Figura 3.2: Fluxograma da pipeline de agendamento e execução de cenas com Dagster.

3.2.3 Desenvolvimento da Interface de Usuário com Streamlit

A interface de visualização e interação com o usuário (frontend) foi desenvolvida integralmente em Python utilizando a biblioteca Streamlit. Conforme justificado na Seção 3.1.1, a escolha desta ferramenta baseou-se na sua capacidade de criar rapidamente aplicações web interativas e ricas em dados, sem a necessidade de desenvolvimento frontend tradicional (HTML, CSS, JavaScript). Isso permitiu que o foco permanecesse na lógica da aplicação e na apresentação eficaz dos dados energéticos e funcionalidades de controle.

A metodologia de desenvolvimento da interface com Streamlit abrangeu os seguintes aspectos técnicos e funcionais:

- **Estruturação Multi-Página e Navegação:** A aplicação foi organizada em um layout multi-página para oferecer uma navegação clara e intuitiva entre as diferentes seções de funcionalidade. Utilizou-se o padrão de seleção de página na barra lateral (`st.sidebar.radio` ou similar) para alternar entre as visualizações principais: "Resumo da Casa", "Detalhes por Dispositivo", "Análise Avançada", "Controle Individual de Dispositivos" e "Gerenciamento de Cenas". Cada página foi encapsulada em sua própria função Python para modularidade.

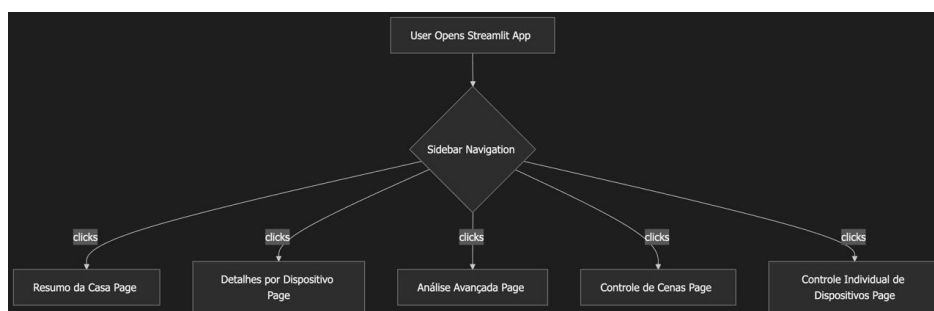


Figura 3.3: Fluxograma da navegação geral da aplicação Streamlit.

- **Carregamento e Filtragem de Dados:** Funções dedicadas em `app/streamlit/utils/data_` foram desenvolvidas para carregar os dados processados da camada de staging (arquivos Parquet em `app/data/staging/`). Estas funções utilizam Pandas para ler os dados e aplicar filtros com base nas seleções do usuário na interface (e.g., intervalo de datas, seleção de dispositivos específicos), disponibilizando DataFrames Pandas para as rotinas de visualização e análise. O cacheamento (`@st.cache_data`) foi empregado em funções de carregamento de dados para otimizar o desempenho, evitando releituras desnecessárias de arquivos.
- **Visualização de Dados Interativa:** Foram implementados diversos tipos de gráficos para apresentar os dados de consumo energético, potência, tensão e cor-

rente. Utilizaram-se as capacidades de plotagem nativas do Streamlit (que por sua vez podem usar bibliotecas como Matplotlib, Plotly ou Vega-Lite internamente) para criar:

- Séries temporais (e.g., `st.line_chart`) para visualizar a evolução do consumo ao longo do tempo.
- Gráficos de barras (e.g., `st.bar_chart`) para comparar consumos entre dispositivos ou períodos.
- Histogramas e gráficos de densidade para analisar a distribuição dos valores de potência, tensão e corrente.
- Métricas chave (e.g., `st.metric`) para destacar valores importantes como consumo total, picos de potência, etc.

A interatividade foi garantida através de widgets do Streamlit como seletores de data (`st.date_input`), seletores de dispositivo (`st.selectbox`, `st.multiselect`), e botões (`st.button`) para acionar recálculos ou atualizações.

- **Desenvolvimento de Funcionalidades de Controle de Dispositivos:**

- **Controle Individual:** Para o controle direto de ligar/desligar dispositivos, foram utilizados widgets `st.toggle`. A interação com estes toggles aciona callbacks que, por sua vez, chamam funções auxiliares (em `app/streamlit/utils/tuya_api_helpers.py`) para enviar os comandos correspondentes à API Tuya. Foi implementado um padrão de "atualização otimista" utilizando o `st.session_state` do Streamlit: o estado visual do toggle é alterado imediatamente na interface, enquanto o comando é enviado à API em background. Em caso de falha na comunicação com a API, o estado do toggle pode ser revertido, e uma mensagem de erro é exibida ao usuário (`st.error`). O uso de `st.fragment` foi adotado para esta seção para otimizar as atualizações da interface, garantindo que apenas o widget do dispositivo alterado seja re-renderizado, melhorando a responsividade.
- **Gerenciamento de Cenas:** Uma interface mais elaborada, utilizando formulários (`st.form`), foi desenvolvida para permitir que os usuários criem, visualizem, modifiquem e excluam cenas de automação. A criação de cena foi dividida em múltiplas etapas (wizard-style) gerenciadas com `st.session_state` para coletar nome da cena, dispositivos envolvidos, ações desejadas (ligar/desligar) e parâmetros de agendamento (horário, dias da semana, recorrência). As definições das cenas são persistidas em um arquivo JSON

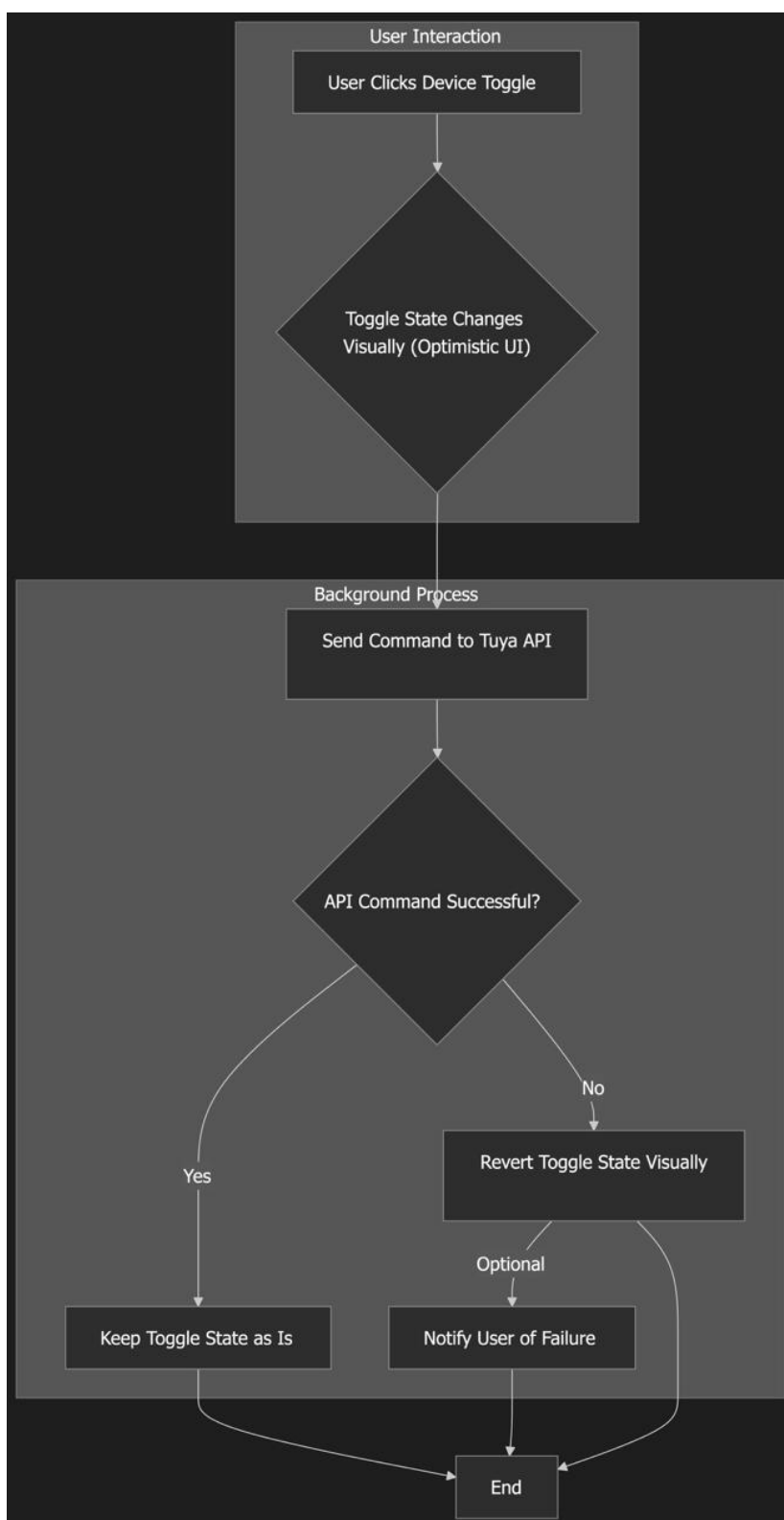


Figura 3.4: Fluxograma da lógica de controle individual de dispositivo na aplicação Streamlit.

(`app/data/scheduled_scenes.json`), que é lido e atualizado pela aplicação Streamlit e também utilizado pelo job Dagster de execução de cenas.

- **Organização e Modularização do Código:** Para manter a clareza e a manutibilidade do código principal da aplicação Streamlit (`app/streamlit/streamlit_app.py`), grande parte da lógica foi refatorada em módulos auxiliares dentro do diretório `app/streamlit/utils/`. Isso incluiu:
 - `data_helpers.py`: Funções para carregar, filtrar e pré-processar dados.
 - `ui_helpers.py`: Funções para componentes de UI reutilizáveis, como o carregamento de CSS customizado e formatação de métricas.
 - `page_functions.py`: Funções que renderizam o conteúdo de cada página da aplicação.
 - `tuya_api_helpers.py`: Funções para interagir com a API Tuya (obter status, enviar comandos), com tratamento de erros e caching.
- **Estilização Customizada com CSS:** Um arquivo CSS externo (`app/streamlit/style.css`) foi criado para aplicar estilos personalizados a elementos da interface, como métricas, títulos e espaçamentos, melhorando a aparência visual geral e a experiência do usuário. Este arquivo é carregado na aplicação através de uma função que injeta o conteúdo CSS no HTML da página.

Esta abordagem metodológica para o desenvolvimento da interface permitiu criar uma aplicação web funcional, interativa e visualmente agradável, utilizando exclusivamente o ecossistema Python e as facilidades oferecidas pelo Streamlit.

3.3 Implementação de Algoritmos de Processamento e Análise

Para o tratamento dos dados coletados, foram desenvolvidos e implementados algoritmos específicos, detalhados a seguir:

3.3.1 Pré-processamento dos Dados

Os dados brutos coletados pelas tomadas passaram por etapas de limpeza, transformação e normalização, visando garantir a qualidade e a consistência necessárias para as análises subsequentes. Este pré-processamento incluiu:

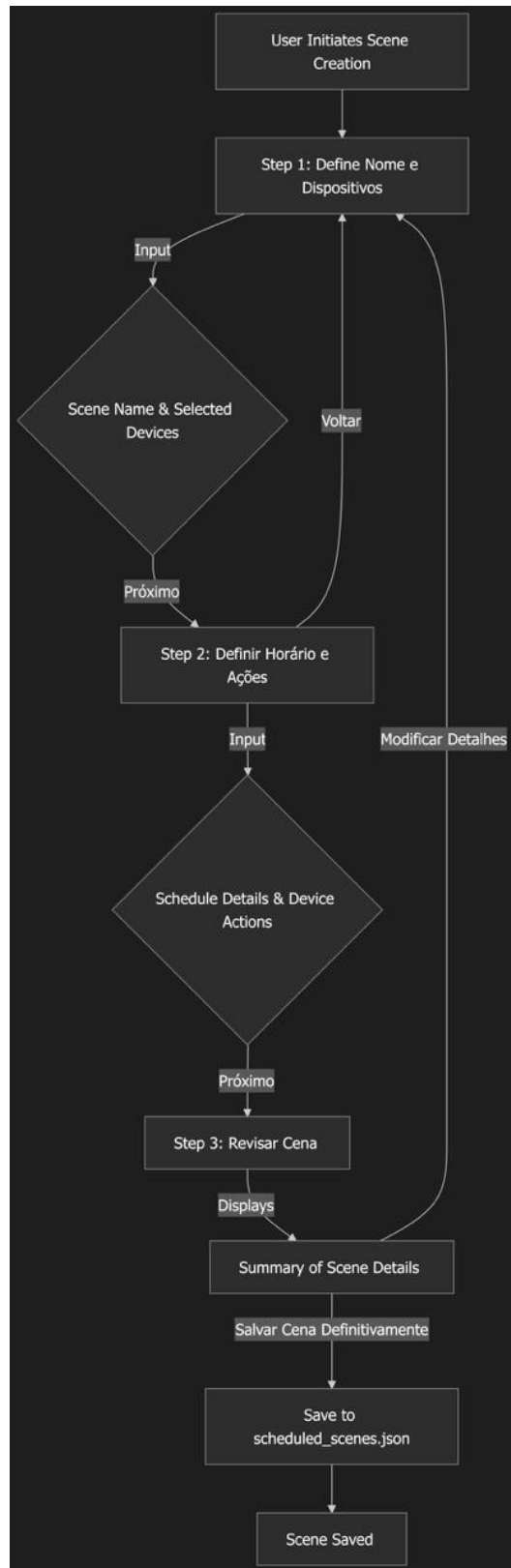


Figura 3.5: Fluxograma do processo de criação de cenas na aplicação Streamlit.

- **Tratamento de Leituras Espúrias:** Identificação e tratamento de dados anômalos, como valores de potência ou corrente negativos ou excessivamente elevados (considerados fisicamente implausíveis para os dispositivos monitorados), que foram removidos ou ajustados com base em limiares predefinidos e análise de outliers.
- **Normalização e Consistência:** Verificação da uniformidade das unidades de medida e tratamento de possíveis dados faltantes, utilizando interpolação linear para pequenas lacunas temporais ou remoção de períodos com ausência significativa de dados, quando aplicável.
- **Enriquecimento dos Dados:** Adição de informações contextuais, como identificação do dispositivo e timestamp padronizado.

Parte fundamental deste pré-processamento é realizada automaticamente pelo ativo `staging_tuya_logs` da pipeline Dagster. Este ativo converte tipos de dados (e.g., timestamp de milissegundos para formato de data e hora padrão), enriquece os registros com informações de mapeamento de dispositivos (nome do dispositivo, local, etc.) provenientes do arquivo `app/data/device_mapping.json`, e estrutura os dados em um formato Parquet particionado e otimizado para análises temporais eficientes.

3.3.2 Análise Estatística e Controle Estatístico de Processos (CEP)

O Controle Estatístico de Processo (CEP), ou Statistical Process Control (SPC), é uma metodologia robusta fundamentada em princípios estatísticos, utilizada para monitorar, controlar e, idealmente, otimizar processos. O objetivo primordial do CEP é a redução da variabilidade inerente a qualquer processo, levando a uma melhoria na qualidade e previsibilidade dos resultados. Uma distinção fundamental no CEP é feita entre dois tipos de variação:

- **Variação por Causas Comuns (ou Aleatórias):** É a variabilidade natural e inerente a um processo que está estatisticamente sob controle. É resultado de múltiplas causas pequenas e inevitáveis.
- **Variação por Causas Especiais (ou Atribuíveis):** Resulta de eventos ou circunstâncias específicas que não são parte intrínseca do processo. Estas causas são geralmente esporádicas e podem ser identificadas e eliminadas.

O CEP utiliza ferramentas gráficas, principalmente os gráficos de controle, para distinguir entre esses dois tipos de variação. Um gráfico de controle plota uma estatística

do processo (e.g., média, amplitude, proporção) ao longo do tempo, em relação a uma linha central (Center Line - CL), um Limite Superior de Controle (Upper Control Limit - UCL) e um Limite Inferior de Controle (Lower Control Limit - LCL). Um processo é considerado estatisticamente sob controle se os pontos plotados variarem aleatoriamente dentro desses limites, sem padrões não aleatórios. Se pontos caem fora dos limites ou exibem padrões específicos, isso indica a presença de uma causa especial de variação, sinalizando que o processo está fora de controle e requer investigação.

Para a detecção de variações significativas e anomalias no consumo energético residencial, a principal técnica de CEP empregada neste trabalho foi o gráfico de Soma Acumulada (CUSUM). Os gráficos CUSUM são particularmente eficazes na detecção de pequenas mudanças persistentes na média de um processo, às quais os gráficos de Shewhart tradicionais (como $\bar{X} - R$ ou XmR) podem ser menos sensíveis. A força do CUSUM reside em sua capacidade de incorporar informações de toda a sequência de observações passadas, em vez de avaliar cada ponto de dados isoladamente.

A implementação do CEP com gráficos CUSUM neste projeto seguiu uma abordagem estruturada, com uma base matemática e estatística detalhada a seguir:

- **Fase I: Estabelecimento dos Parâmetros de Controle (Linha de Base):**

Antes do monitoramento prospectivo (Fase II), é crucial estabelecer uma linha de base representativa do processo operando sob controle. Para cada dispositivo ou agregado de consumo energético a ser monitorado, coletou-se um conjunto de m observações históricas ($X_1, X_2, \dots, X_m$) consideradas como representativas da operação normal. Assumindo que a métrica de interesse (e.g., consumo diário de energia em kWh, potência média horária em W) X_j segue uma distribuição aproximadamente normal quando o processo está sob controle, com média μ_0 e desvio padrão σ_0 , estas são estimadas a partir dos dados da Fase I:

- Estimativa da média do processo sob controle ($\hat{\mu}_0$):

$$\hat{\mu}_0 = \bar{X} = \frac{1}{m} \sum_{j=1}^m X_j \quad (3.1)$$

- Estimativa do desvio padrão do processo sob controle ($\hat{\sigma}_0$):

$$\hat{\sigma}_0 = s = \sqrt{\frac{1}{m-1} \sum_{j=1}^m (X_j - \bar{X})^2} \quad (3.2)$$

Para dados individuais, como leituras de consumo em intervalos de tempo, esta estimativa de s é apropriada. Em alguns contextos de CEP, se os dados

são agrupados em subgrupos, outras estimativas de σ_0 (e.g., baseadas na amplitude média $\bar{R}$) podem ser usadas.

Estes parâmetros $\hat{\mu}_0$ e $\hat{\sigma}_0$ são então utilizados como os valores de referência μ_0 e σ_0 para a Fase II de monitoramento.

- **Fase II: Monitoramento com Gráficos CUSUM Tabulares:** Com os parâmetros μ_0 e σ_0 estabelecidos, o monitoramento de novas observações X_i (para $i = 1, 2, \dots$ na Fase II) é realizado utilizando duas estatísticas CUSUM unilaterais para detectar aumentos ou diminuições na média do processo.

- **CUSUM Superior (S_{Hi}):** Projetado para detectar um aumento na média do processo. É calculado recursivamente para cada nova observação X_i :

$$S_{Hi} = \max(0, X_i - (\mu_0 + K) + S_{H(i-1)}) \quad (3.3)$$

onde $S_{H0} = 0$ (o CUSUM é inicializado em zero). O valor K é um parâmetro de referência, frequentemente chamado de folga (allowance ou slack value). A lógica é que S_{Hi} acumula desvios positivos de X_i em relação a um valor alvo ajustado $\mu_0 + K$. Se X_i é menor ou igual a $\mu_0 + K$, a soma acumulada não aumenta ou é resetada para zero se o termo $X_i - (\mu_0 + K) + S_{H(i-1)}$ for negativo.

- **CUSUM Inferior (S_{Li}):** Projetado para detectar uma diminuição na média do processo. É calculado recursivamente:

$$S_{Li} = \max(0, (\mu_0 - K) - X_i + S_{L(i-1)}) \quad (3.4)$$

onde $S_{L0} = 0$. Similarmente, S_{Li} acumula desvios quando X_i cai abaixo de um valor alvo ajustado $\mu_0 - K$.

- **Seleção dos Parâmetros do CUSUM (K e H):** A performance de um gráfico CUSUM é determinada pela escolha de dois parâmetros: K (o valor de referência ou folga) e H (o limite de decisão).

- **Valor de Referência (K):** O parâmetro K é a magnitude do desvio da média alvo μ_0 que é considerado suficientemente pequeno para não ser acumulado. É tipicamente expresso em termos do desvio padrão do processo, $K = k \cdot \sigma_0$. Um valor comum para o fator k é 0.5. Esta escolha torna o gráfico CUSUM ótimo para detectar rapidamente um desvio na média do

processo de magnitude $\Delta = 2K = 2k\sigma_0$. Por exemplo, se $k = 0.5$, o CUSUM é otimizado para detectar um desvio de $1\sigma_0$ na média. O valor K nas Equações 3.3 e 3.4 representa esta folga.

- **Limite de Decisão (H):** O parâmetro H é o limite de controle. Um sinal de fora de controle é gerado se S_{Hi} exceder H ou se S_{Li} exceder H . Assim como K , H também é frequentemente expresso em termos do desvio padrão, $H = h \cdot \sigma_0$. Valores comuns para o fator h variam tipicamente de 4 a 5.

A escolha dos fatores h e k (e, conseqüentemente, H e K) é um compromisso que afeta o Comprimento Médio de Corrida (Average Run Length - ARL) do gráfico. O ARL_0 é o número médio de amostras antes de um falso alarme (quando o processo está, de fato, sob controle), enquanto o ARL_1 é o número médio de amostras necessárias para detectar uma mudança real de uma determinada magnitude na média do processo. Valores maiores de H aumentam o ARL_0 (menos falsos alarmes) mas também aumentam o ARL_1 (detecção mais lenta de mudanças reais). No contexto deste projeto, os valores de K e H foram ajustados empiricamente com base na análise dos dados históricos de cada dispositivo e na sensibilidade desejada, visando um equilíbrio prático entre a detecção de anomalias no consumo energético e a ocorrência de alarmes espúrios.

- **Regra de Sinalização e Ação:** O processo é considerado fora de controle e uma investigação é acionada se:
 - $S_{Hi} > H$: Indica um aumento significativo e persistente na média do consumo energético.
 - $S_{Li} > H$: Indica uma diminuição significativa e persistente na média do consumo energético.

Após um sinal, é prática comum resetar o valor do CUSUM que excedeu H de volta para zero (ou para um valor de "headstart", embora zero seja comum para CUSUMs tabulares) e investigar a causa especial. A magnitude da inclinação no gráfico CUSUM antes do sinal pode também ser usada para estimar a nova média do processo.

Implementação Prática e Visualização no Streamlit: No sistema desenvolvido, a análise de CEP utilizando gráficos CUSUM foi integrada à interface Streamlit na página "Análise Avançada". Nesta seção, o usuário pode selecionar um dispositivo específico e o período de interesse para análise. Com base nos dados históricos e nos parâmetros de controle (μ_0, σ_0, K, H) definidos para o dispositivo (conforme descrito

na Fase I e na seleção de parâmetros), a aplicação calcula e exibe dinamicamente os gráficos CUSUM superior (S_{Hi}) e inferior (S_{Li}) para a métrica de consumo energético selecionada (e.g., potência média diária). Os gráficos são renderizados utilizando a biblioteca Plotly, permitindo interatividade como zoom e visualização de valores pontuais. Os limites de decisão H são plotados como linhas horizontais nos gráficos. Quando um valor de S_{Hi} ou S_{Li} cruza seu respectivo limite H , um ponto de "fora de controle" é indicado visualmente no gráfico. Adicionalmente, um registro de "fault logs" (registros de falha) é apresentado, listando os timestamps e as magnitudes dos desvios que levaram à sinalização, auxiliando o usuário na identificação e diagnóstico de anomalias no consumo do dispositivo. Os parâmetros K e H foram, para esta implementação, definidos com base na análise empírica dos dados históricos de cada tipo de dispositivo, mas a arquitetura permite a futura inclusão de interfaces para que usuários avançados possam ajustar esses parâmetros, se necessário.

Esta abordagem de CEP, utilizando gráficos CUSUM, foi complementada por métodos estatísticos descritivos para caracterizar os padrões de consumo de cada dispositivo e do agregado residencial. Estes métodos incluíram o cálculo de médias, medianas, desvios padrão, valores mínimos e máximos, e a visualização de dados através de histogramas, boxplots e séries temporais para identificar tendências, sazonalidades e a distribuição geral do consumo energético. A combinação dessas técnicas permitiu uma análise mais robusta e detalhada dos padrões de consumo e a identificação de comportamentos atípicos.

3.4 Validação em Ambiente Real

O protótipo foi submetido a uma fase de validação em um ambiente residencial real, caracterizado pela ausência de infraestrutura prévia de automação, permitindo uma avaliação do sistema em condições de uso típicas. Esta etapa foi crucial para aferir a aplicabilidade e eficácia da solução proposta e envolveu os seguintes aspectos:

- **Avaliação de Desempenho dos Sensores e Algoritmos:**
 - *Precisão das Medições:* A acurácia das tomadas inteligentes *Tuya Smart Wi-Fi* foi avaliada comparando suas leituras de potência instantânea e consumo acumulado com as de um medidor de energia calibrado, conectado em série com os aparelhos monitorados. Foram realizados testes com diferentes tipos de carga (resistiva, indutiva, eletrônica) para verificar a consistência das medições.

- *Eficiência dos Algoritmos de CEP*: A capacidade dos algoritmos de CEP, especialmente os gráficos CUSUM, em detectar anomalias foi testada através da simulação de eventos controlados (e.g., introdução de um aparelho defeituoso, alteração significativa no padrão de uso de um dispositivo) e da observação da rapidez e precisão da sinalização pelo sistema. A eficiência computacional, em termos de tempo de processamento para atualização dos gráficos e alertas, também foi monitorada.

- **Análise de Usabilidade da Interface Streamlit:**

- A interface de usuário desenvolvida com Streamlit foi avaliada por um grupo restrito de usuários-teste, representativos do público-alvo (residentes sem conhecimento técnico aprofundado em análise de dados energéticos).
- A avaliação focou na clareza da apresentação dos dados (gráficos, métricas), na facilidade de navegação entre as diferentes seções da aplicação ("Resumo da Casa", "Detalhes por Dispositivo", etc.), na intuitividade das funcionalidades de controle de dispositivos e no gerenciamento de cenas.
- O feedback foi coletado por meio de observação direta da interação dos usuários com o sistema e através de um questionário simplificado, abordando aspectos como facilidade de aprendizado, satisfação com a interface e utilidade percebida das informações fornecidas.

- **Teste de Robustez e Confiabilidade do Sistema:**

- O sistema foi operado continuamente por um período estendido (e.g., algumas semanas) para monitorar sua estabilidade e resiliência em condições reais de uso.
- Foram observadas as respostas do sistema a eventos como interrupções na conexão Wi-Fi, falhas temporárias na API Tuya (simuladas ou ocorridas naturalmente), e reinicializações dos dispositivos de hardware e do servidor da aplicação.
- A capacidade de recuperação automática da pipeline de dados Dagster e do agendador de cenas, bem como a integridade dos dados armazenados após tais eventos, foram pontos chave desta avaliação.

Os resultados detalhados de cada teste de validação foram sistematicamente documentados. Este conjunto de informações foi fundamental para identificar pontos de melhoria, corrigir inconsistências encontradas e refinar o comportamento geral do sistema, visando otimizar sua eficiência, confiabilidade e a experiência do usuário final.

3.5 Viabilidade de Implementação em Larga Escala

A análise de viabilidade considerou aspectos técnicos, econômicos e sociais, como:

- **Custos de produção:** Avaliação dos investimentos necessários para replicação do sistema.
- **Benefícios econômicos:** Potencial de redução de custos para os usuários com base na diminuição do desperdício energético.
- **Impactos ambientais:** Contribuição para a sustentabilidade por meio do uso eficiente de energia.
- **Estratégias de adoção:** Identificação de barreiras e propostas para ampliar o alcance do sistema, como parcerias com fabricantes de dispositivos inteligentes.

3.6 Resumo do Capítulo

Este capítulo detalhou as etapas metodológicas seguidas no desenvolvimento do sistema inteligente para monitoramento e controle estatístico de processo (CEP) para o monitoramento de energia residencial. As etapas compreenderam a análise de tecnologias, desenvolvimento do protótipo utilizando as tomadas *Tuya Smart Wi-Fi*, implementação de algoritmos avançados, validação experimental e estudo de viabilidade em larga escala. A abordagem proposta combina rigor técnico e aplicabilidade prática, destacando-se como uma solução promissora para o setor de automação residencial.

Capítulo 4

Resultados

Para a execução do projeto, algumas etapas de desenvolvimento tiveram de ser seguidas: familiarização com o sistema, estudo dos módulos envolvidos, leitura dos requisitos, elaboração de documento descrevendo todo o processo de implementação e relacionamento com os diversos módulos, implementação e testes.

4.1 Atividades do Projeto

4.2 Requisitos do Sistema

4.3 Desenvolvimento e Implementação

A Tabela 4.1 apresenta as atividades executadas.

4.4 Testes

4.5 Análise dos Resultados

Apresente os resultados sem adulterações e faça análises objetivas. Pense na melhor maneira de apresentar os resultados graficamente. Se os gráficos são difíceis de inter-

Atividade 1	aa a	ab b
Ativ. 2	aa a	ab b

Tabela 4.1: Exemplo de tabela - Coloque toda informação sobre a tabela aqui

pretar, talvez tabelas sejam uma forma melhor de apresentar resultados. Não apresente dados (gráficos e tabelas) se não há uma conclusão interessante a ser tirada. Lembre-se de ser conciso.

Não se esqueça das unidades! Pense que *a priori* todo número deve ter uma unidade. Não escreva as unidades em itálico (no ambiente matemático) e tome cuidado para diferenciar maiúsculas e minúsculas. Um exemplo é escrever 22 [kN] e não 22KN (Kelvin vezes Newton!).

Ao apresentar resultados experimentais, tome o cuidado para também apresentar o cálculo das incertezas sempre que forem significativas. Ao fazer conclusões, sempre considere se o tamanho da sua amostra é grande o suficiente do ponto de vista estatístico. Lembre que a média empírica $\hat{\mu}_X$ de N observações independentes da variável X_i possui variância

$$\hat{\sigma}_\mu^2 = \frac{1}{N(N-1)} \sum_{i=1}^N (X_i - \hat{\mu}_X)^2 ,$$

onde se assume que as variáveis X_i possuem uma mesma distribuição e que essa distribuição possui segundo momento finito.

4.6 Resumo do Capítulo

Tente não terminar de forma abrupta. Se for escrever algo aqui, não seja genérico!

4.7 Formato, expressões matemáticas e o L^AT_EX

4.7.1 O L^AT_EX

O L^AT_EX é o método preferencial de preparação de documentos para textos técnicos nas ciências exatas. O L^AT_EX permite não só lidar com equações de uma forma mais prática que em editores de texto, mas também facilita a formatação de documentos e tem um desempenho marcadamente superior a editores de texto na preparação de documentos longos como monografias.

Documentos em L^AT_EX são escritos em um ou mais arquivos de texto com extensão .tex. Após a escrita, o .tex é *compilado* para gerar arquivos nos formatos .pdf, .dvi ou .ps. Hoje há duas distribuições padrão para o L^AT_EX. Sistemas Windows usam o MikT_EX e sistemas Unix usam o T_EXLive. Além das distribuições, muitos usuários utilizam *front-ends* que facilitam a edição do texto, a compilação e a instalação de pacotes.

Os pacotes necessários para compilar o presente documento devem ser encontrados

numa instalação completa dessas distribuições. Se tiver dificuldades com os pacotes, você pode instalá-los manualmente ou tentar alterar o código para usar versões antigas dos mesmos.

A compilação pode ser feita pelos comandos `latex` ou `pdflatex`, invocados pela linha de comando ou pelo *front-end*. Note que será necessário empregar o comando **mais de uma vez** para que as referências cruzadas saiam corretas.

Como discutido na Seção ??, uma ferramenta útil para gerenciar as citações em L^AT_EX é o BibT_EX. Para gerar uma lista bibliográfica a partir do arquivo `.bib`, este arquivo deve ser indicado no arquivo `.tex`. Em seguida devem-se executar os comandos `pdflatex`, `bibtex` e `pdflatex` novamente sempre usando o `.tex` como argumento. Note que os comandos são executados nesta ordem e de forma repetida para que as referências cruzadas sejam geradas corretamente.

Nesta seção você deve encontrar exemplos dos comandos mais usados em L^AT_EX. Outros exemplos e manuais podem ser encontrados na internet com facilidade.

4.7.2 Expressões Matemáticas

Ao escrever expressões matemáticas, defina todas as variáveis antes de usá-las ou imediatamente depois da expressão. Deixar de fazê-lo torna seu texto **ilegível**. Segue um exemplo.

Seja o par $(a_1, a_2) \in \mathbb{R}^2$. Para $s \in \mathbb{C}$, definimos a função $f(s)$ como

$$f(s) \triangleq \frac{a_1 s + a_2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \ ,$$

onde os escalares $\zeta, \omega_n > 0$ são constantes.

Note que não foi necessário atribuir valores às variáveis neste momento. Repare também como devemos **usar pontuação** (vírgula) nas equações, tratando-as como parte da frase. Usamos o símbolo $\triangleq$ ou $:=$ para deixar explícito que se trata de uma definição. Ser claro nesse aspecto facilita o entendimento do leitor.

A equação acima não foi numerada porque não será citada no texto. Vejamos um exemplo com numeração.

A função $f(\cdot)$ possui um zero em $-a_2/a_1$ (ou $-\frac{a_2}{a_1}$) e, para $\zeta < 1$, possui polos complexos $p_{1,2}$ dados por

$$p_{1,2} = \omega_n \left(-\zeta \pm j\sqrt{1 - \zeta^2} \right) \ . \quad (4.1)$$

Agora podemos citar os polos dados pela Equação (4.1) (aqui adotamos a convenção de citar sempre com o número entre parênteses precedido da palavra Equação). Note

como usamos um comando especial na Equação 4.1 para garantir o ajuste automático do tamanho dos parênteses.

Vejamos agora como criar equações alinhadas. Considere o sistema dinâmico dado pelas equações diferenciais:

$$\dot{x}_1 = \cos(x_2) \cdot \ln(1/x_1) + \tan(u) \quad (4.2)$$

$$\dot{x}_2 = e^{-x_1-x_2}$$

$$y = \min\{x_1, x_2\} \quad , \quad (4.3)$$

onde $x(t) = [x_1(t) \ x_2(t)]'$, $t > 0$, é a variável de estado do sistema, $u(t)$ é o sinal de entrada e $y(t)$ é o sinal de saída do sistema. Note no .tex que o caracter de tabulação & foi usado para indicar o ponto de alinhamento horizontal das equações. Além disso, para ilustrar o uso do L^AT_EX, retiramos a numeração da segunda equação e citamos as equações separadamente.

Nas Equações (4.2) e (4.3), aparecem operadores como min, ln, cos e tan. A convenção aqui é que **variáveis devem ser escritas em itálico e operadores não**. Por essa razão todas as expressões matemáticas devem ser escritas no ambiente matemático (entre cifrão) mesmo quando for possível usar texto comum. Isso garante a consistência das fontes utilizadas (nem sempre a fonte do ambiente matemático é a mesma fonte do texto).

Para escrever matrizes, podemos fazer por exemplo:

$$\sum_{n=0}^{\infty} z^{-n} \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}^n = \begin{bmatrix} \frac{z}{z-\lambda} & \frac{z}{(z-\lambda)^2} \\ 0 & \frac{z}{z-\lambda} \end{bmatrix}, \quad \forall \lambda < |z| \quad .$$

Para escrever uma expressão com múltiplos casos, podemos fazer, para um inteiro N positivo,

$$g[n] = \begin{cases} 0, & \text{se } n \leq 0 \\ n, & \text{se } n = 1, 2, \dots, N-1 \\ N, & \text{se } n \bmod N = 0 \\ 0, & \text{caso contrário} \end{cases} .$$

Nunca reaproveite símbolos matemáticos, isto é, nunca use o mesmo símbolo para designar variáveis diferentes.

Para um exemplo com múltiplas linhas de expressão matemática: tem-se que, para $a \neq 0$,

$$\begin{aligned} ax^2 + bx + c &= 0 \\ \Rightarrow a(x^2 + bx/a + c/a) &= 0 \Rightarrow a((x + b/(2a))^2 + c/a - b^2/(4a^2)) = 0 \\ \Rightarrow (x + b/(2a))^2 &= (b^2 - 4ac)/(4a^2) \\ \Rightarrow (x + b/(2a)) &= \pm\sqrt{b^2 - 4ac}/(2a) \\ \Rightarrow x &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} . \end{aligned} \tag{4.4}$$

Note a argumentação lógica aqui. Não estamos dizendo que o valor de x é dado pela última linha. Estamos dizendo que a hipótese da primeira linha juntamente com a hipótese $a \neq 0$ implicam os referidos valores de x . **Um erro comum dos alunos ao escrever é não distinguir a veracidade das implicações com a veracidade das hipóteses.**

Capítulo 5

Conclusões

Novamente, este será um dos trechos que o leitor experiente lerá antes de decidir se vale a pena ler o texto integral. Seja convincente.

5.1 Considerações Finais

Reitere o que de mais importante foi feito, qual era o objetivo inicial e qual o resultado obtido. Se houve requisitos ou especificações de projeto, discuta se foram atingidos. Se os resultados não foram conclusivos ou contrariam o que se esperava, seja honesto e diga-o explicitamente. Busque explicar os insucessos com argumentos sólidos e plausíveis.

5.2 Propostas de Continuidade

Se houve questões ainda não respondidas ou resultados insatisfatórios, aponte direções de continuação.

Apêndice A

O que ficou para depois

Inclua aqui informações que não sejam tão relevantes para o entendimento do projeto mas que ainda sejam importantes para documentá-lo.

Apêndice B

O que mais faltou

Inclua aqui informações que não sejam tão relevantes para o entendimento do projeto mas que ainda sejam importantes para documentá-lo.

Referências Bibliográficas

- [1] BOX, G. E. P.; JENKINS, G. M. *Time Series Analysis - Forecasting and Control*. San Francisco: Holden Day, 1976.
- [2] BRIGHAM, E. O. *The Fast Fourier Transform*. Englewood Cliffs: Prentice-Hall, 1974.
- [3] EGGERT, R. J. Response tables from industry members of ASME's design engineering division and the manufacturing engineering division - Engineering design education: Surveys of demand and supply. *Boise State University*, 2002.
- [4] SPRETNAK, C. M. A survey of the frequency and importance of technical communication in an engineering career. *Technical Writing Teacher*, ERIC, v. 9, n. 3, p. 133–36, 1982.
- [5] KRETH, M. L. A survey of the co-op writing experiences of recent engineering graduates. *Professional Communication, IEEE Transactions on*, IEEE, v. 43, n. 2, p. 137–152, 2000.
- [6] CUNNINGHAM, D.; STEWART, J. Perceptions and practices: A survey of professional engineers and architects. *ISRN Education*, Hindawi Publishing Corporation, v. 2012, 2012.